

监督和无监督学习《计算与人工智能概论》讲过，不写了

线性回归模型：

分类中只有少数可能的输出，是离散数据集合；回归可以有任意输出
细节中学都讲过，不讲了

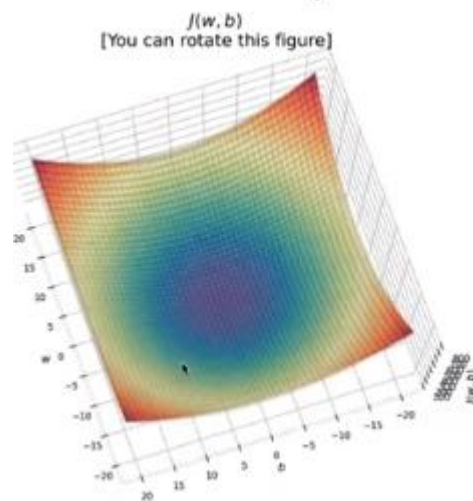
代价函数：

平方误差成本函数 J：

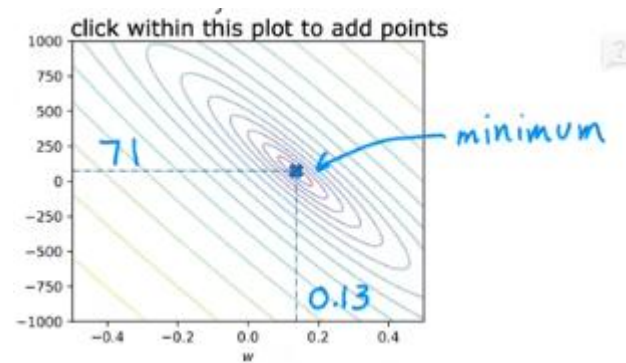
$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m \left(\underset{\text{error}}{\hat{y}^{(i)} - y^{(i)}} \right)^2$$

m = number of training examples

用成本函数衡量线性回归 $f = wx + b$ 的 w 和 b ，目标是找到 w 和 b 让 J 最小化



除了用三维图将 J 可视化，还可以使用等高线图



梯度下降：

以线性回归为例。设置 w 和 b 的初始值，不断改变 w, b 去减小 J ，直到 J 接近最小值

J 可能存在多个极小值

一座山有多个山谷。站在山上，环顾四周，找到下山最快的路线然后走一步，再环顾四周，如此反复。但是这样走的话，初始化位置不同，可能走进不同山谷。

这时梯度下降算法可能陷入不同的“最小值”

α 是学习率。重复下面操作不断更新 w, b ，使 J 达到局部最小：

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

这里 tmp_w 和 tmp_b 被赋值给 w, b 进行下一轮更新

学习率：

上面的 α 选择不好，梯度下降可能不起作用

α 太小，下降太慢；太大，下降过程中可能跨过 J 的最低点，永远不能到达最小

如果已经达到局部最小，偏导数是 0，不再下降

随着梯度下降，我们的步会逐渐减小，因为导数变小

用于线性回归的梯度下降：

$$\begin{aligned} \frac{\partial}{\partial w} J(w, b) &= \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (\underline{wx^{(i)} + b} - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cancel{2x^{(i)}} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)} \\ \frac{\partial}{\partial b} J(w, b) &= \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^m (\underline{wx^{(i)} + b} - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cancel{2} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \end{aligned}$$

代入原式，得：

repeat until convergence {

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

Update w and b simultaneously

$\frac{d}{db} J(w, b)$

$f_{w,b}(x^{(i)}) = w x^{(i)} + b$

根据初始的 w, b ，可能会得到不同的局部最小值

如果在凸函数进行梯度下降，学习率合适，始终收敛到全局最小值

多维特征：

$x_j = j^{th}$ feature
 n = number of features
 $\vec{x}^{(i)}$ = features of i^{th} training example
 $x_j^{(i)}$ = value of feature j in i^{th} training example

x 上面带一个箭头，右上角加上 (i) ，表示这是一个列向量

原来的线性回归只考虑一个特征，可以使用多个特征：

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

也可以用行向量 $w = (w_1 \ w_2 \ \dots \ w_n)$ ， $x = (x_1 \ x_2 \ \dots \ x_n)^T$ ， $f = w \cdot multiply(x)$ 表示

Parameters and features

$\vec{w} = [w_1 \ w_2 \ w_3]$ $n=3$

b is a number

$\vec{x} = [x_1 \ x_2 \ x_3]$

linear algebra: count from 1

NumPy

$w = \text{np.array}([1.0, 2.5, -3.3])$

$b = 4$ $x[0] \ x[1] \ x[2]$

$x = \text{np.array}([10, 20, 30])$

code: count from 0

Without vectorization $n=100,000$

$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$

$f = w[0] * x[0] +$
 $w[1] * x[1] +$
 $w[2] * x[2] + b$



DOT 的函数或方法

Without vectorization

$f_{\vec{w},b}(\vec{x}) = \left(\sum_{j=1}^n w_j x_j \right) + b$ $\sum_{j=1}^n \rightarrow j=1 \dots n$
 $1, 2, 3$

$\text{range}(0, n) \rightarrow j = 0 \dots n-1$

$f = 0$ $\text{range}(n)$
 for j in $\text{range}(0, n)$:
 $f = f + w[j] * x[j]$
 $f = f + b$



Vectorization

$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

$f = \text{np.dot}(w, x) + b$

矢量化可以简化程序，运行更快

用于多元线性回归的梯度下降法:

	Previous notation	Vector notation
Parameters	$w_1, \dots, w_n$ b	$\vec{w} = [w_1 \dots w_n]$ (vector of length n) b still a number
Model	$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + \dots + w_n x_n + b$	$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
Cost function	$J(\underbrace{w_1, \dots, w_n}_\text{weights}, b)$	$J(\underbrace{\vec{w}}_\text{weights}, b)$ (dot product)
Gradient descent	repeat { $w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\underbrace{w_1, \dots, w_n}_\text{weights}, b)$ $b = b - \alpha \frac{\partial}{\partial b} J(\underbrace{w_1, \dots, w_n}_\text{weights}, b)$ }	repeat { $w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$ $b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$ }

Gradient descent

The diagram is divided into two sections by a vertical line. The left section is titled 'One feature' and shows a 'repeat' loop containing the weight update formula: $\underline{w} = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \underline{x^{(i)}}$. A pink box highlights the term $\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$, with a pink arrow pointing to the partial derivative $\frac{\partial}{\partial w} J(w, b)$. Below this is the bias update formula: $b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$. The right section is titled ' n features ($n \geq 2$)' and shows a 'repeat' loop for $j=1$ to $j=n$. For each j , the weight update formula is: $\underline{w_j} = w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\bar{w},b}(\bar{x}^{(i)}) - y^{(i)}) \underline{x_j^{(i)}}$. A pink box highlights the term $\frac{1}{m} \sum_{i=1}^m (f_{\bar{w},b}(\bar{x}^{(i)}) - y^{(i)}) x_j^{(i)}$, with a pink arrow pointing to the partial derivative $\frac{\partial}{\partial w_j} J(\bar{w}, b)$. Below this is the bias update formula: $b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\bar{w},b}(\bar{x}^{(i)}) - y^{(i)})$. At the bottom right, there is a play button icon.

与原来的线性回归相比，特征变多，需要更新更多的 w

用正规方程法也可以求解。但是特征数量太多的时候算的太慢，大多数都是用梯度下降

特征缩放:

例如，用面积和卧室数量估计房价，前者范围很大，后者很小。一般来说，一个特征的可能值比较小，那对应的参数会比较大；如果可能值比较大，对应参数就比较小

如果说它们的参数在同一量度，那么前者的改变会对预测结果造成较大影响，后者影响则太小。需要对训练数据进行一些转换，让它们具有可比较的范围。也就是数据归一化：

均值归一化: u 是均值, 分母是 $\max(x) - \min(x)$, 变成 $(0, 1)$ 的正态分布

$$x_1 = \frac{x_1 - \mu_1}{2000 - 300}$$

Z-score 标准化: μ 是均值, σ 是标准差。最后变成均值 0 标准差 1 的分布

$$x_1 = \frac{x_1 - \mu_1}{\sigma_1}$$

如果范围显著不同，那就要进行归一化。一个是（0，3），另一个是（-2，0.5），这时可以进行归一化，但不进行归一化一般也可以正常工作；但另一个是（-100，100），范围差距太大，需要归一化；另一个是（-0.001，0.001），这个范围太小，也需要缩放。缩放后梯度下降运行更快。如果数据不能有负数或者有异常值，均值归一最好不用。

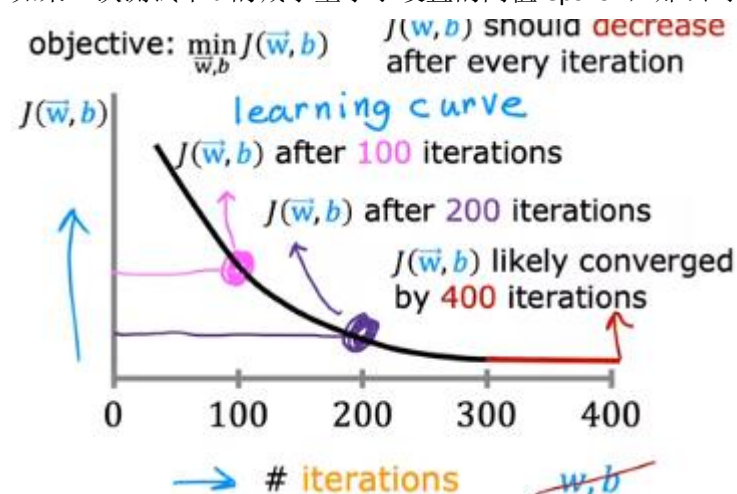
判断梯度下降是否收敛：

如果梯度下降正常工作，成本函数J应该减小，如果增加，可能是学习率或代码逻辑有问题。

如果最后J几乎不减小，说明这时收敛了。

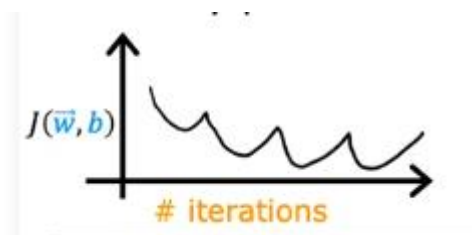
很难提前知道要几次迭代后才梯度收敛，所以需要借助图像或自动收敛测试。

如果一次测试中J的减小量小于设置的阈值 epsilon，那么可以声明收敛。一般看图表。

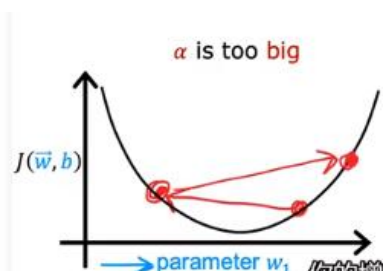


如何设置学习率：

太小的话运行太慢，太大可能不会收敛。



这种情况下可能是学习率太大，无法收敛，要使用较小的学习率。

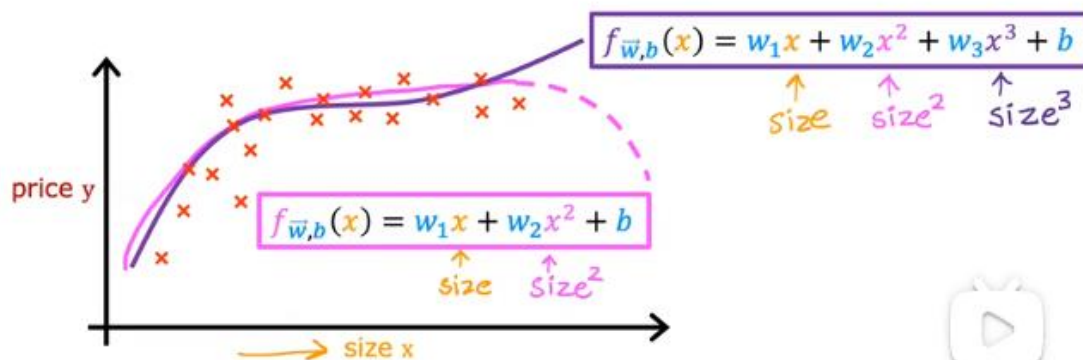


在足够小的学习率下，成本函数 J 应该每次都减小
 α 应该每次都设为一个比较小的数
 这里可以多次尝试找到合适的学习率，比如 0.001, 0.01, 0.1 等
 尝试过程中要尽量找到一个比合理值略小的 α

特征工程：通过组合原来的特征得到一个新特征，可能更适合计算。比如，将房子的宽度等特征转换成面积这个新特征来预测房屋价格

多项式回归：
 这个可以用来拟合非线性函数，比如用房子大小来预测房价时：

Polynomial regression



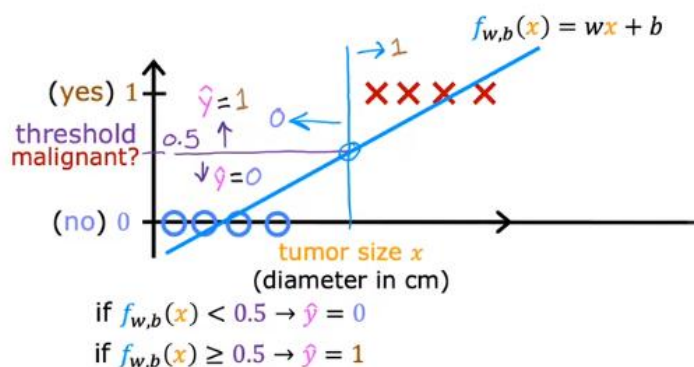
这时，特征缩放非常重要。当房子大小范围显著不同，梯度下降可能出现问题

$$f_{\bar{w},b}(x) = w_1x + w_2\sqrt{x} + b$$

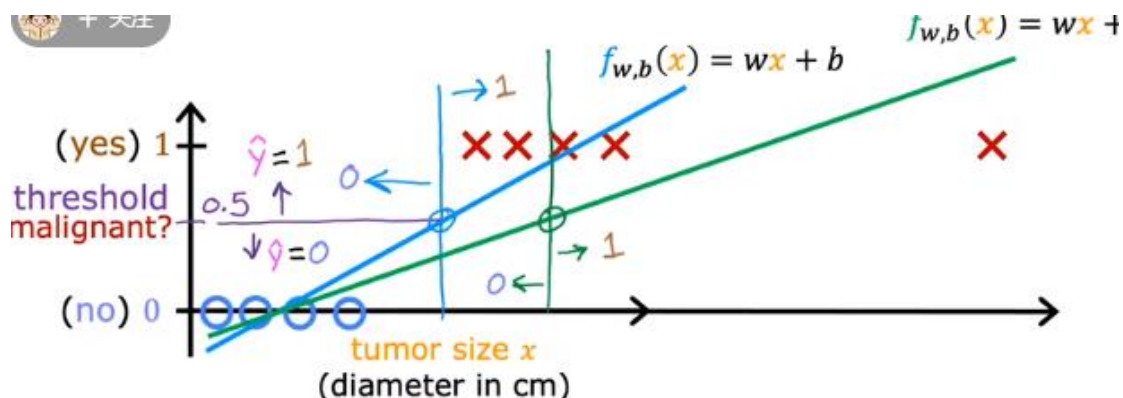
$\uparrow$ size $\uparrow$ $\sqrt{\text{size}}$

变量 x 的指数可以是任意实数，取决于根据数据分布，我们做出什么选择

动机与目的：
 用 0 表示 False，1 表示 True（好的一类）进行分类



线性回归并不是很适合分类。用线性回归进行分类，0.5 为阈值，在右边的点就归为正类，

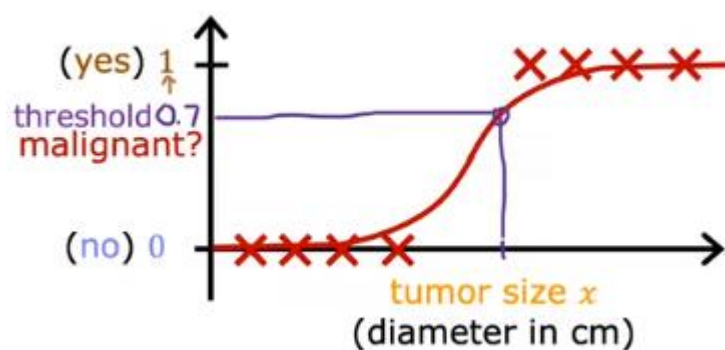


在这种情况下，最右边的正类导致决策边界右移影响分类结果

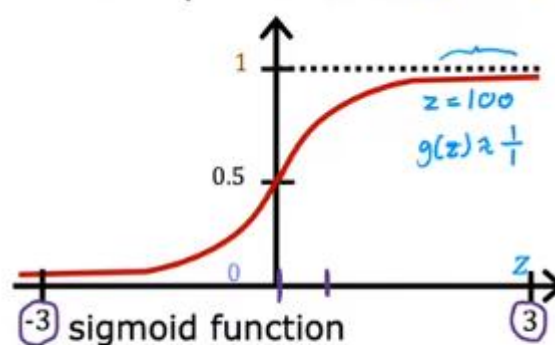
逻辑回归：

拟合一条这样的曲线进行分类：

假设有一点的预测值是 0.7，但是最终要输出分类结果 0 或 1，这时要用到逻辑函数 sigmoid



Want outputs between 0 and 1



logistic function

outputs between 0 and 1

$$g(z) = \frac{1}{1+e^{-z}} \quad 0 < g(z) < 1$$

当 z 是一个比较大的数，sigmoid 函数接近 1.用该函数构建逻辑回归：

原来的决策函数 $f(x)$ 变成：

$$f_{\vec{w},b}(\vec{x})$$

$$z = \vec{w} \cdot \vec{x} + b$$

$$g(z) = \frac{1}{1+e^{-z}}$$

$$f_{\vec{w},b}(\vec{x}) = g(\underbrace{\vec{w} \cdot \vec{x} + b}_z) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x} + b)}}$$

"logistic regression"

用这种方法进行预测得到 0.7，则说明有 70%的可能是 1，有 30%的机会是 0
 通过决策边界可以更好地看出分类结果
 分类时需要设置一个阈值，预测结果大于该阈值，归类为 1

When is $f_{\vec{w},b}(\vec{x}) \geq 0.5$?

$$g(z) \geq 0.5$$

$$z \geq 0$$

$$\vec{w} \cdot \vec{x} + b \geq 0$$

$$\vec{w} \cdot \vec{x} + b < 0$$

线性化 现在讨论

$$\hat{y} = 1$$

$$\hat{y} = 0$$

下面有两个特征，当位于线的右边，分类预测为 1

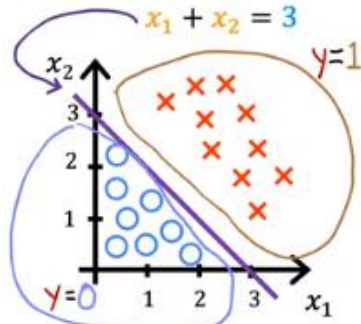
Decision boundary

$$f_{\vec{w},b}(\vec{x}) = g(z) = g(\underbrace{w_1 x_1 + w_2 x_2 + b}_{\substack{1 \quad 1 \quad -3}})$$

Decision boundary $z = \vec{w} \cdot \vec{x} + b = 0$

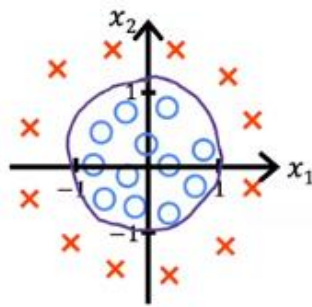
$$z = x_1 + x_2 - 3 = 0$$

$$x_1 + x_2 = 3$$



决策边界可以是非线性。下面，当数据点在园外，归类为 1

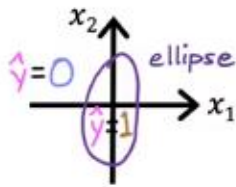
Non-linear decision boundaries



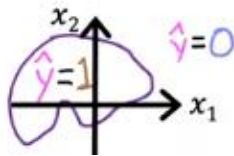
$$f_{\vec{w},b}(\vec{x}) = g(z) = g\left(\frac{w_1 x_1^2 + w_2 x_2^2 + b}{1}\right)$$

decision boundary $z = x_1^2 + x_2^2 - 1 = 0$
 $x_1^2 + x_2^2 = 1$

当我们有不同的特征，分类可能会更复杂



$$f_{\vec{w},b}(\vec{x}) = g(z) = g(w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1 x_2 + w_5 x_2^2 + w_6 x_1^3 + \dots + b)$$



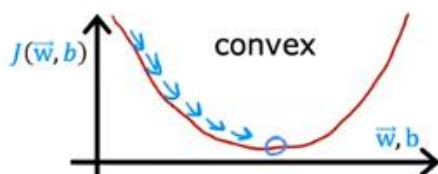
逻辑回归中的代价函数：

给定 $f(x)$ ，怎么选择其中的 w 和 b ？

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$$

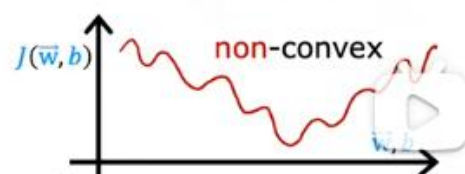
linear regression

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$



logistic regression

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$



把线性回归的代价函数放在逻辑回归中不合适，这不是凸函数，不一定收敛到全局最小

$$\text{cost } J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$$

loss $L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$

引入损失函数，保证 f 是凸函数

Logistic loss function

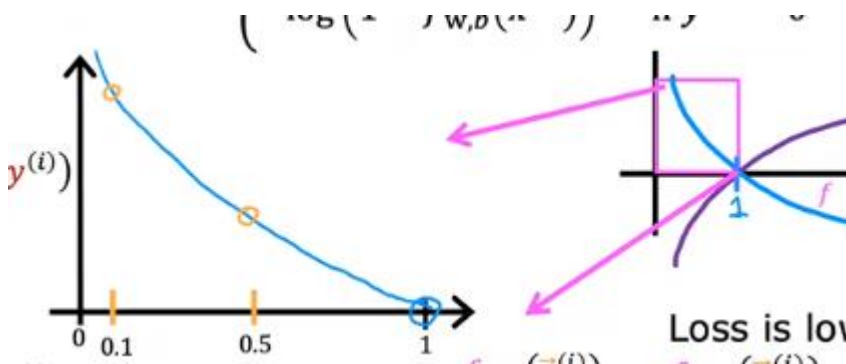
$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\bar{w},b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\bar{w},b}(\bar{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

现在输出的 f 始终保持在 0 和 1 之间，通过标签 y 是 0 还是 1 决定损失函数

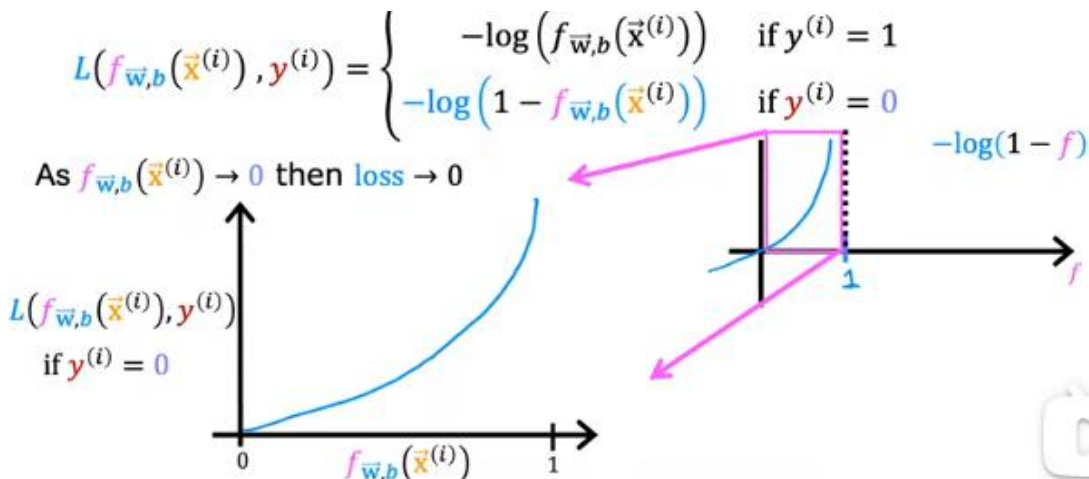
损失函数用于衡量训练数据时所有的损失

如果算法预测值接近 1，真实标签为 1，那么损失很小。如果算法预测为 0.5，存在损失，但相对没有那么高。相反，如果算法输出 0.1，即有 10% 的可能性是 0，那么损失在这里的价值就高得多。当 $y=1$ 时损失函数激励或帮助算法做出准确预测，因为损失最低

$y=1$ 的情况如下图：



如果 $y=0$ ：



f 接近 0 时损失也很小， f 值越大损失也越大

上面的损失函数可以简化为：

$$L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)}) = -y^{(i)}\log(f_{\bar{w},b}(\bar{x}^{(i)})) - (1 - y^{(i)})\log(1 - f_{\bar{w},b}(\bar{x}^{(i)}))$$

成本函数变成：

$$\begin{aligned} J(\bar{w}, b) &= \frac{1}{m} \sum_{i=1}^m [L(f_{\bar{w},b}(\bar{x}^{(i)}), y^{(i)})] \\ &= \frac{1}{m} \sum_{i=1}^m [y^{(i)}\log(f_{\bar{w},b}(\bar{x}^{(i)})) + (1 - y^{(i)})\log(1 - f_{\bar{w},b}(\bar{x}^{(i)}))] \end{aligned}$$

实现梯度下降:

cost

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) \right]$$

repeat {
 $w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$
 $b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

repeat {
 $w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right]$
 $b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) \right]$
 } simultaneous updates

looks like linear regression!

Same concepts:
 • Monitor gradient descent (learning curve)

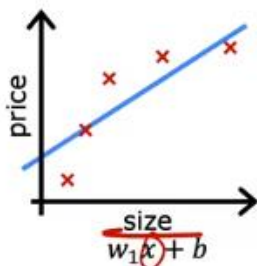
Linear regression $f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

Logistic regression $f_{\vec{w}, b}(\vec{x}) = \frac{1}{1 + e^{(-\vec{w} \cdot \vec{x} + b)}}$

与线性回归相似，可以用梯度下降提高逻辑回归的效率。梯度下降中，可以进行矢量化（变成数组，用 numpy 提高运算效率）和特征缩放。

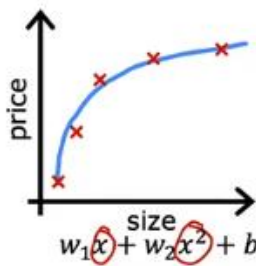
过拟合问题:

Regression example

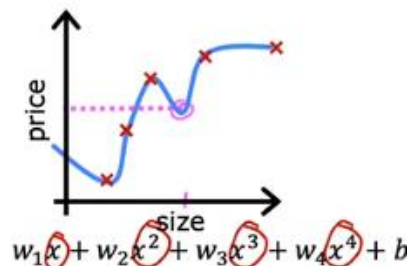


underfit

- Does not fit the training set well



- Fits training set pretty well

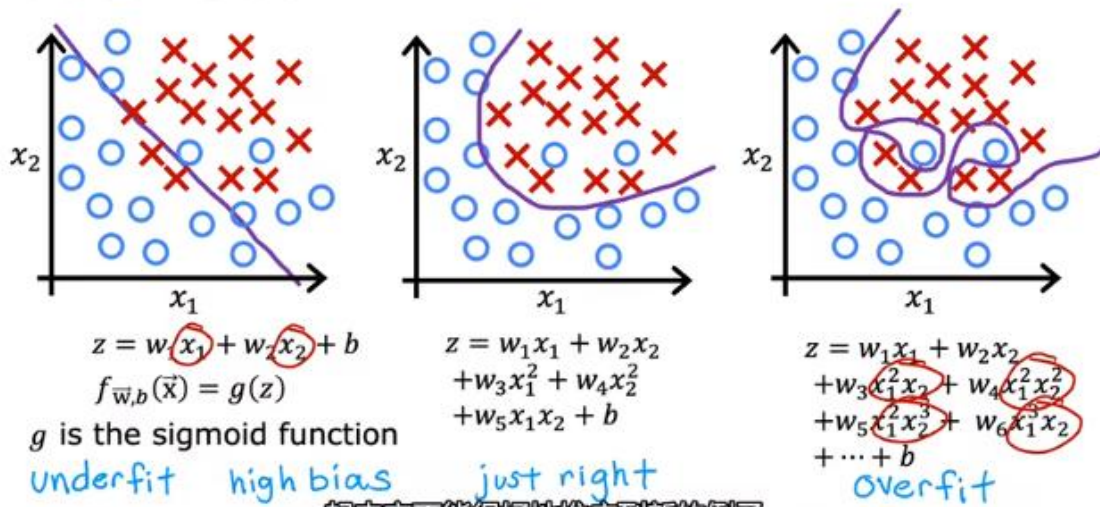


overfit

- Fits the training set extremely well

第一种拟合效果不佳，有高偏差，认为是欠拟合；第二种是我们期望的拟合效果；第三种完美通过所有点，但不太适合这里的数据，有高方差，是过拟合。过拟合的模型误差小，但记住了训练集的噪声，无法推广到新数据集
 同样，过拟合也存在于分类模型

Classification

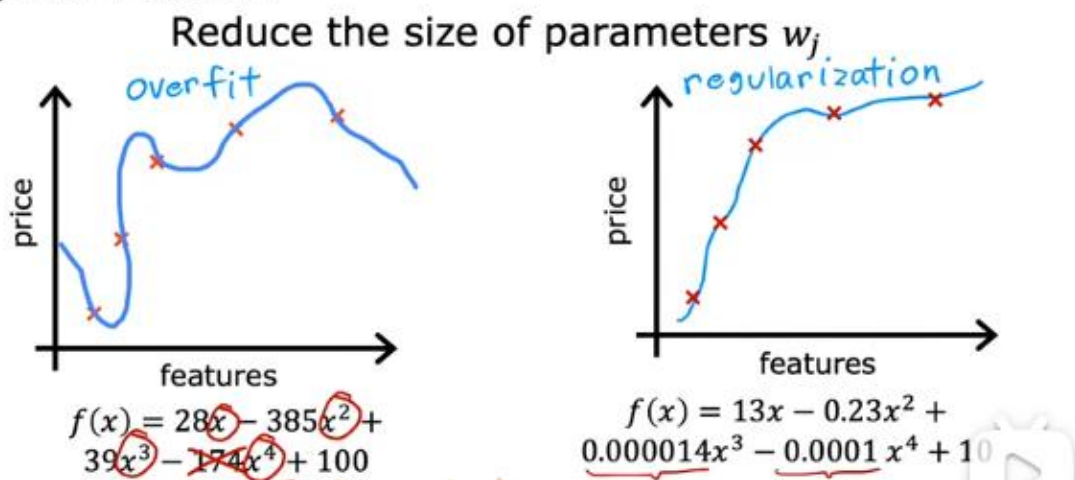


三种分别是欠拟合、适当的拟合、过拟合

解决过拟合:

1. 使用更多的训练数据
2. 使用更少的功能。比如，多项式拟合时，降低多项式的次。也就是特征选择。选择与因变量相关性最大的几个特征，但是可能丢失一些信息
3. 正则化

正则化鼓励算法减小某个参数，而不是把它直接设置为 0。比如用高阶多项式拟合，有的参数可能很大，可以令其接近 0，比如设为 0.001



通过正则化保留所有特征，修改参数避免它们产生过大的影响

正则化:

不知道哪个参数重要，于是惩罚所有特征，想办法让代价函数最小

在原来的代价函数中引入一个新项，得到新的函数，用于对特征进行惩罚

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

这个函数也叫正则化函数

正则化一般只有 w 而不含 b 。这个算法想办法让 w 较小

狼塔 λ 太大，模型会欠拟合；如果太小，容易过拟合。 λ 恰到好处时拟合最好
正则化用于线性回归时：

$$\min_{\vec{w}, b} J(\vec{w}, b) = \min_{\vec{w}, b} \left[\frac{1}{2m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \right]$$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

} simultaneous update

对新的代价函数进行梯度下降，得到合适的 w 和 b

上面式子可以简化为：

repeat {

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m [(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}] + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

} simultaneous update $j=1 \dots n$

$$w_j = \underbrace{w_j \left(1 - \alpha \frac{\lambda}{m}\right)}_{\text{usual update}} - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

α 通常比较小，比如 0.01， λ 可以设置为 1.再除以数据量 m ，下面是可能的结果：

$$\alpha \frac{\lambda}{m} = 0.01 \frac{1}{50} = 0.0002$$

$$w_j (1 - 0.0002)$$

这种情况下 w_j 会一点点收缩

How we get the derivative term (opt

$$\begin{aligned}
 \frac{\partial}{\partial w_j} J(\vec{w}, b) &= \frac{\partial}{\partial w_j} \left[\frac{1}{2m} \sum_{i=1}^m \underbrace{(f(\vec{x}^{(i)}) - y^{(i)})^2}_{\vec{w} \cdot \vec{x}^{(i)} + b} + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \right] \\
 &= \frac{1}{2m} \sum_{i=1}^m \left[(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)}) \cancel{2x_j^{(i)}} \right] + \frac{\lambda}{2m} \cancel{2} w_j \\
 &= \frac{1}{m} \sum_{i=1}^m \left[\underbrace{(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)})}_{f(\vec{x})} x_j^{(i)} \right] + \frac{\lambda}{m} w_j \\
 &= \frac{1}{m} \sum_{i=1}^m \left[(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j
 \end{aligned}$$

正则化用于逻辑回归:

可以修改逻辑回归的代价函数进行正则化:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

通过引入惩罚函数防止 w 太大。即使特征很多，也能得到合适的拟合曲线

$$\min_{\vec{w}, b} J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left[(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j$$

$j = 1 \dots n$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

}

Looks same as for linear regression!

logistic regression

与线性回归一样，正则化时只针对 w 而不对 b

神经元和大脑:

神经网络中，神经元接受一个或多个输入（数字的形式），经过一系列计算输出一个数
传统的逻辑回归等技术无法有效处理大量数据，神经网络可以有效处理，取决于其规模大小

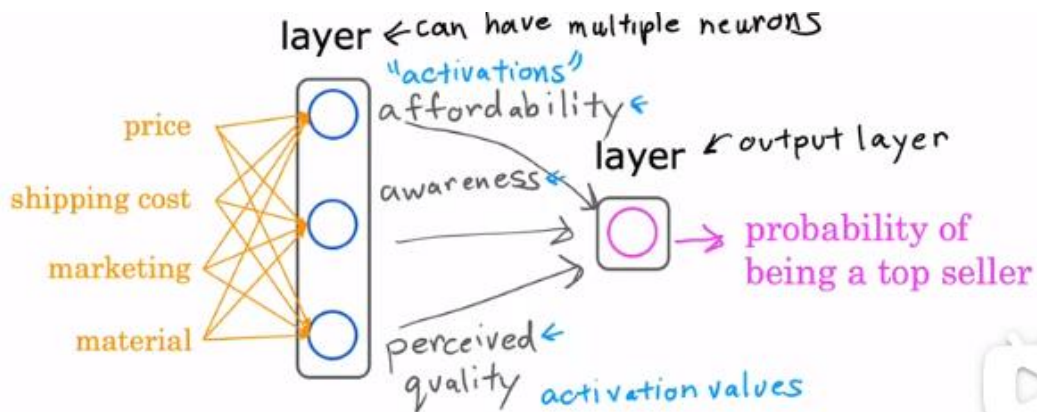
需求预测:

例如，通过输入价格，对一件衣服是否畅销进行分类

传统的逻辑回归可以看作单个神经元，输出概率

神经网络取多个神经元进行预测。

创建一个人工神经网络，通过价格、材料等多个特征判断一件衣服是否实惠
多个特征输入多个神经元，将输出连接到另一个神经元，输出最终概率



一层可以有多个神经元，也可以只有一个。

最右边那层输出结果，叫输出层

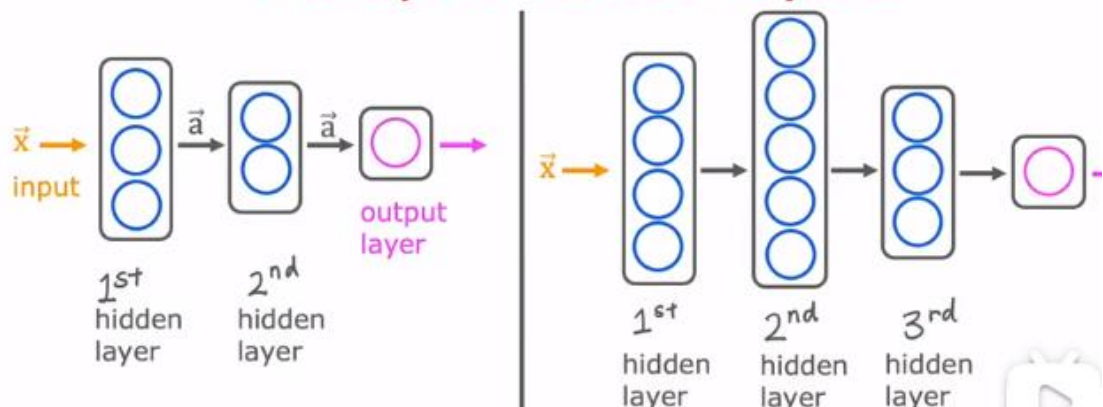
左边三个神经元构成一层，可输入相同或相似的特征

神经元在左边激活，显示在右边。

左边三个神经元输出的数字是激活值，通过这三个数字计算最终的一个输出

输入层可以有多个特征，组成一个特征向量，输入到隐含层，输出三个数字表示一个激活向量，到输出层进行最后的激活，输出结果

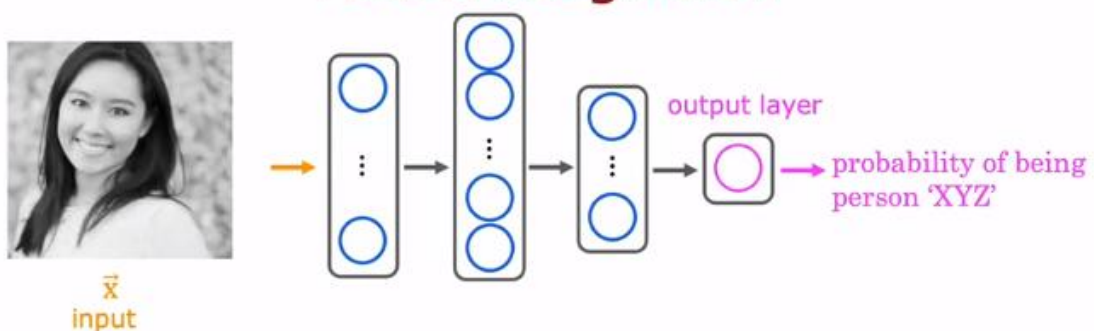
Multiple hidden layers



隐含层有很多种，层数和各层神经元都可以不同，构建神经网络需要决定有几个隐藏层、分别有几个神经元

例如，在图像感知中：

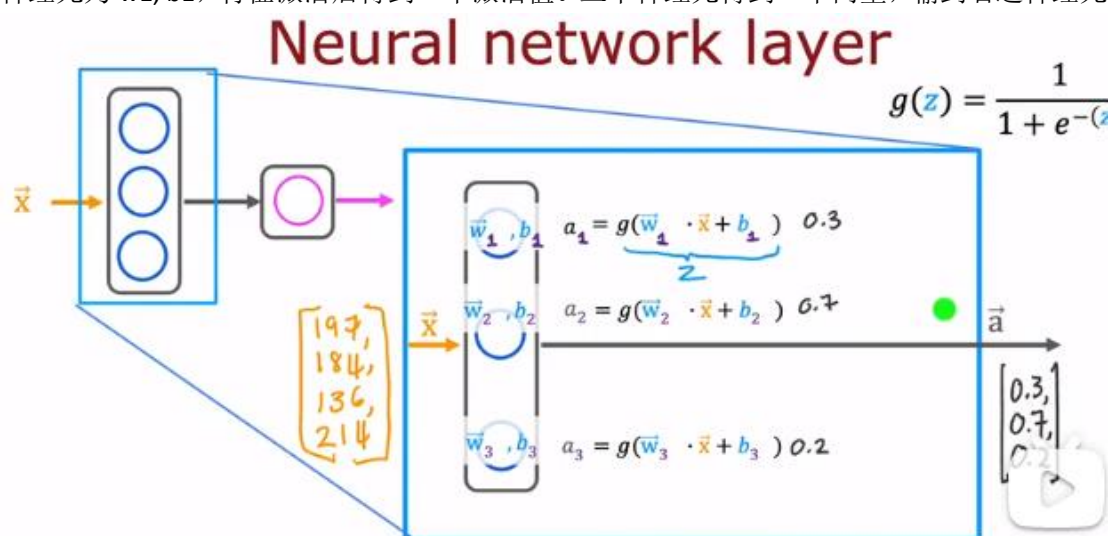
图像灰度化得到一个矩阵，提取特征并输入神经网络，实现人脸识别



比如，第一个隐藏层中，第一个神经元寻找图像的垂直边缘，第二个寻找定向的边缘，第三个寻找那个方向上的线，以此类推；到了第二个隐藏层，尝试找到鼻子等部位，第三个隐藏层完成脸型，最终完成识别

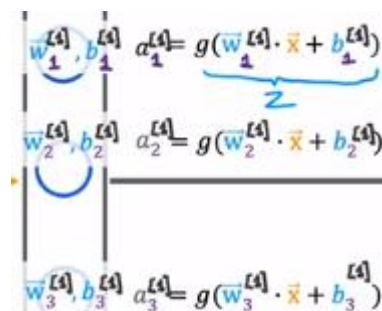
神经网络中的网络层：

比如，下面，一个神经元进行一次逻辑回归，每个神经元都有不同的逻辑回归参数。第一个神经元为 w_1, b_1 ，特征激活后得到一个激活值。三个神经元得到一个向量，输到右边神经元

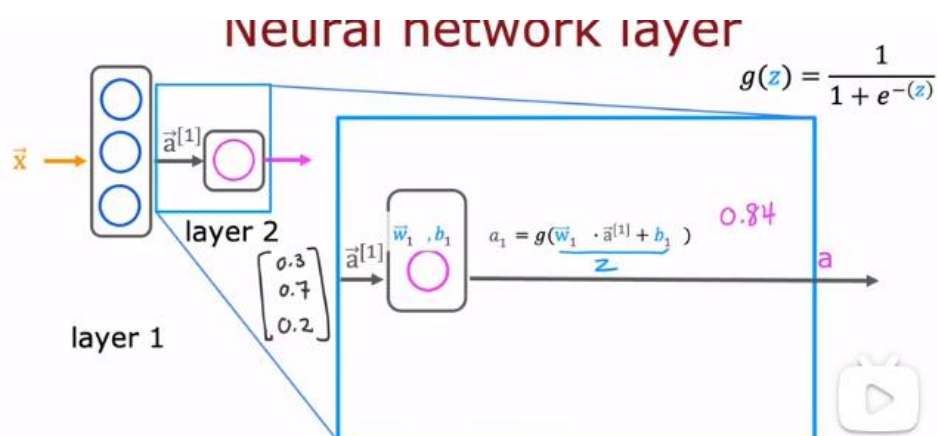


输入层，也就是向量 x ，有时也叫神经网络的第 0 层，向右分别是第一和第二层。有的神经网络可以有几十几百层

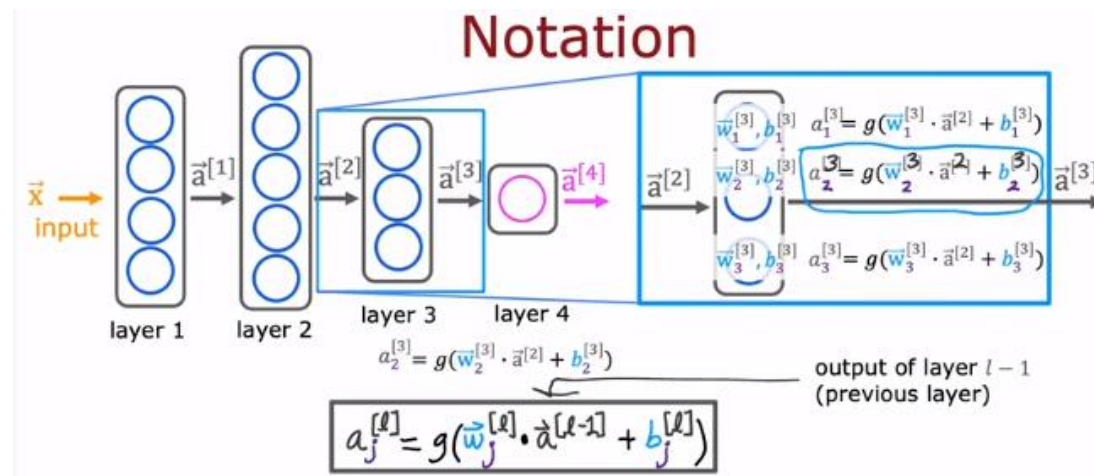
在 w_1 等参数右上方加一个方括号。如果是 [1]，表示与神经网络第一层相关的数量，如果是 2，那就是第二层，以此类推



然后传到第二层，输出结果。比如设阈值是 0.5，输出的概率跟阈值比较，输出分类结果



因为输出层只有一个神经元，所以输出一个标量而不是向量
更复杂的神经网络与之类似：



本层的参数与上一层得到的数据，激活
神经网络的前向传播：就是上面那个神经网络输入数据后左往右推

模型训练细节：

如果是解决回归问题而不是分类问题，可以使用损失函数，与逻辑回归中的成本函数不同。
如果要想误差平方最小化，可以使用均方误差 MSE 作为损失函数。在 tensorflow 中，这叫二元交叉熵损失函数

如果用梯度下降确定神经网络的参数，需要计算 J 的偏导，这用到神经网络的反向传播

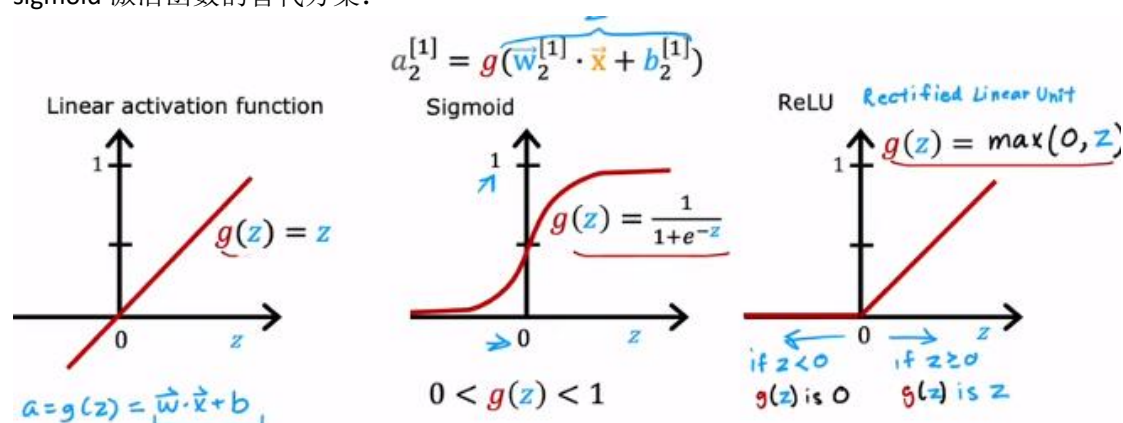
repeat {

$$w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\bar{w}, b)$$

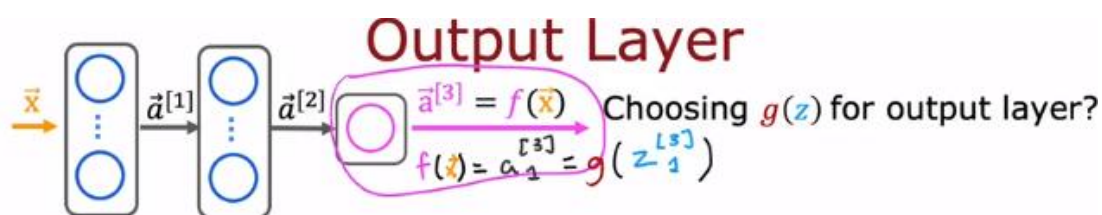
$$b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b} J(\bar{w}, b)$$

在 tensorflow 中通过 fit 实现，输入数据 x, y 和要迭代的次数 epochs

sigmoid 激活函数的替代方案：



上面三个是最常用的激活函数



对于二分类问题，通常用 **sigmoid** 激活函数，因为要预测 y 是 1 的概率。回归问题可能选择不同的激活函数，比如预测股价与今天相比变化的比例，可以使用线性激活函数，因为输出结果可正可负。如果输出的 y 只能是非负数，可以使用 **ReLU** 激活函数，因为这个函数只能是 0 或正数

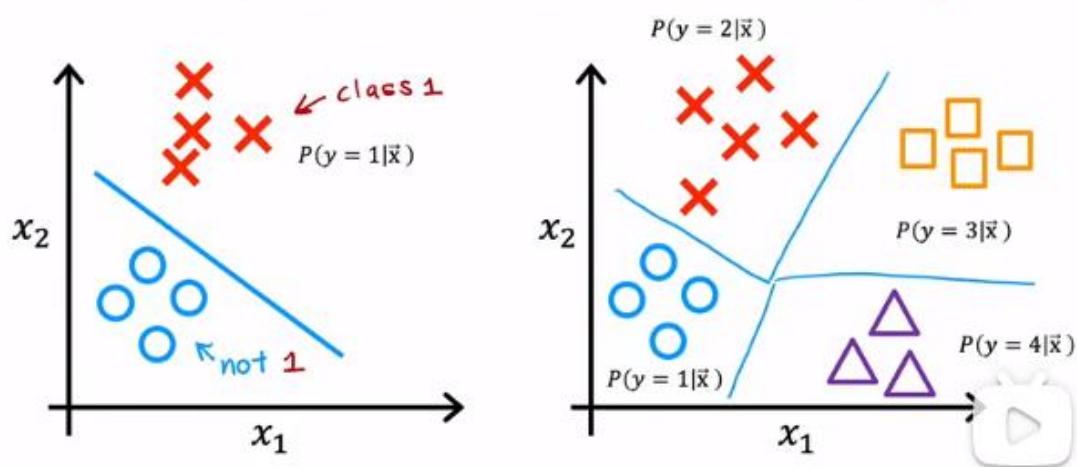
ReLU 的计算效率比 **sigmoid** 高，因为 **sigmoid** 要进行取幂等运算。而且 **ReLU** 只在负数区间平坦，而 **sigmoid** 在图像左右两侧都平展，可能会影响梯度下降的效率。事实证明 **ReLU** 运行更快，比 **sigmoid** 更常用

为什么模型需要激活函数？神经网络中，如果每个隐藏层都用线性激活函数，那最终的效果等同于线性回归；如果前面是线性激活函数，输出层用 **sigmoid**，那就等同于逻辑回归。因此神经网络需要不同的激活函数，而不是全是线性激活函数。通常建议使用 **ReLU**

多分类问题：

取两个以上的离散数据作为分类标签，比如 1, 2, 3, 4. 决策边界相对二分类问题有所不同

Multiclass classification example



Softmax:

如果分类问题中有多个输出，比如 4 个，可以用 **softmax** 回归

Logistic regression
(2 possible output values)
 $z = \vec{w} \cdot \vec{x} + b$

$$\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \quad 0.71$$

$$\circ a_2 = 1 - a_1 = P(y=0|\vec{x}) \quad 0.29$$

Softmax regression
(N possible outputs) $y=1, 2, 3, \dots, N$

$$z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \dots, N$$

parameters $w_1, w_2, \dots, w_N$
 $b_1, b_2, \dots, b_N$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j|\vec{x})$$

Softmax regression (4 possible outputs) $y=1, 2, 3, 4$

$$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1 \quad a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=1|\vec{x}) \quad 0.30$$

$$\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2 \quad a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=2|\vec{x}) \quad 0.20$$

$$\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3 \quad a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=3|\vec{x}) \quad 0.15$$

$$\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4 \quad a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y=4|\vec{x}) \quad 0.35$$

这里 a_1 加到 a_n 和为 1。softmax 回归可以看作逻辑回归的泛化

Cost

Logistic regression

$$z = \vec{w} \cdot \vec{x} + b$$

$$a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x})$$

$$a_2 = 1 - a_1 = P(y=0|\vec{x})$$

$$\text{loss} = \underbrace{-y \log a_1}_{\text{if } y=1} - \underbrace{(1-y) \log(1-a_1)}_{\text{if } y=0}$$

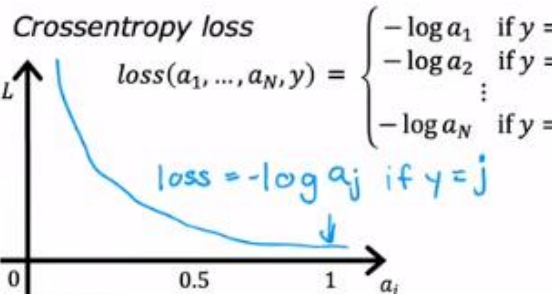
$$J(\vec{w}, b) = \text{average loss}$$

Softmax regression

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y=1|\vec{x})$$

$$\vdots$$

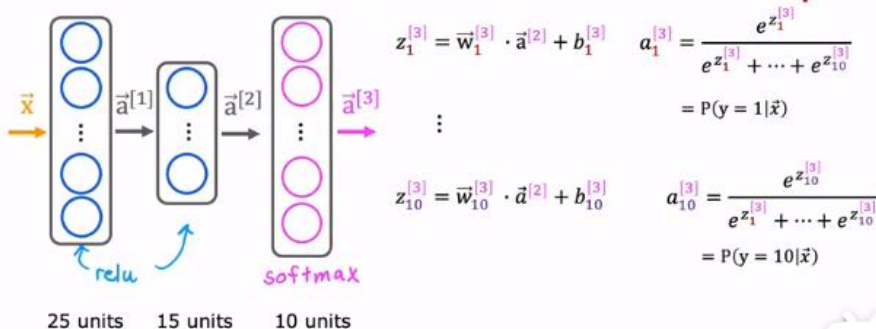
$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y=N|\vec{x})$$



如果 a_j 很接近 1，那么损失就越小。算法鼓励 a_j 尽可能接近 1，因为不管 y 实际值是多少，我们希望 y 成为该值的机会尽可能大

神经网络的 softmax 输出：

Neural Network with Softmax output



通过这种方法可以计算输入特征可能是十个类别中的哪一种

softmax 层有时也叫 softmax 激活函数

逻辑回归中, a_1 是关于 z_1 的函数, a_2 是关于 z_2 的函数, 以此类推。但是 softmax 函数的 a_1 是 z_1 一直到 z_{10} 的函数, 每一个激活值都取决于 z 的所有值

logistic regression

$$a_1^{[3]} = g(z_1^{[3]}) \quad a_2^{[3]} = g(z_2^{[3]})$$

softmax

$$\vec{a}^{[3]} = (a_1^{[3]}, \dots, a_{10}^{[3]}) = g(z_1^{[3]}, \dots, z_{10}^{[3]})$$

softmax 的改进实现:

Numerical Roundoff Errors

More numerically accurate implementation of logistic loss:

Logistic regression:

$$a = g(z) = \frac{1}{1 + e^{-z}}$$

Original loss

$$\text{loss} = -y \log(a) - (1 - y) \log(1 - a)$$

More accurate loss (in code)

$$\text{loss} = -y \log\left(\frac{1}{1 + e^{-z}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-z}}\right)$$

logit: z

上面是逻辑回归的二元交叉熵损失与这里使用的神经网络损失函数。相当于把 a 作为一个中间值, 用 z 算出 a

当涉及 softmax 时, 这时数据舍入误差会变得更糟

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

model.compile(loss=SparseCrossEntropy(from_logits=True))

如果当中有一个 z 比 e 小甚至到了负数, 通过重新排列术语, tensorflow 可以避免其中一些非常小的数字。非常大的数字也是一样。因此为损失函数提出更多的女演员计算? (说白了就是方便进行多分类)

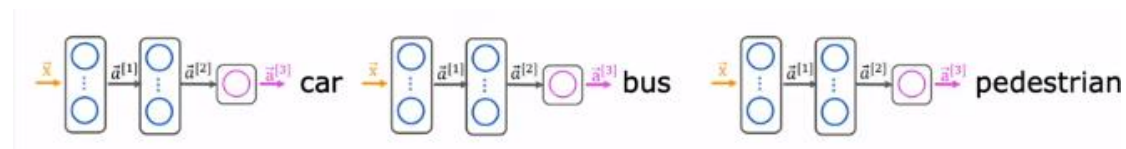
可以理解成把 softmax 交给损失函数进行实现。所以这时上面的 softmax 被划掉改成 linear

激活函数，因为修改了损失函数从而完成了 softmax 的实现

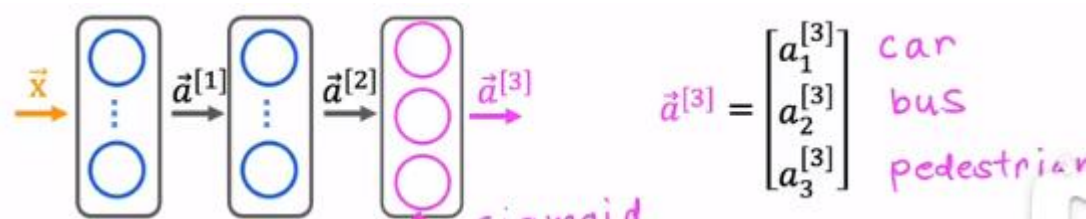
多个输出的分类：

输入一张图片，确定里面是否有汽车、公交车或行人？比如结果是“Y,N,Y”，那就输出一个数组[1 0 1]。

一种方法是把它视为三个完全独立的机器学习问题，建立三个神经网络，分别识别三个目标

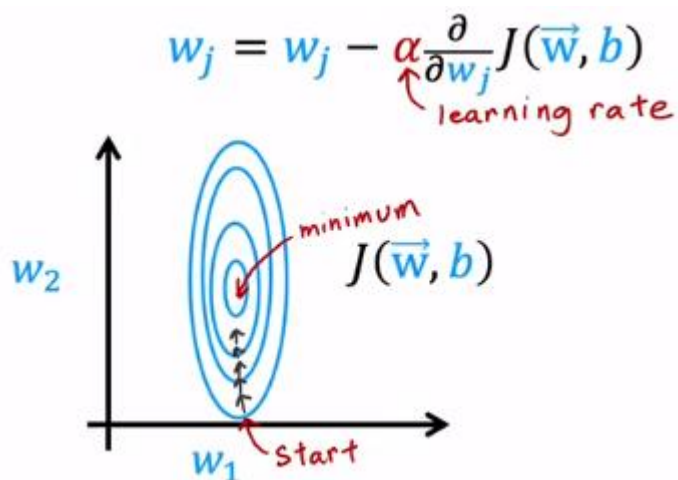


也可以训练单个神经网络，同时检测汽车、公交车和行人



到了这里最右边的隐藏层，可以使用 sigmoid 函数进行二分类，也就是解决三个二分类问题，输出一个数组表示三个东西分别是否存在

高级优化方法：



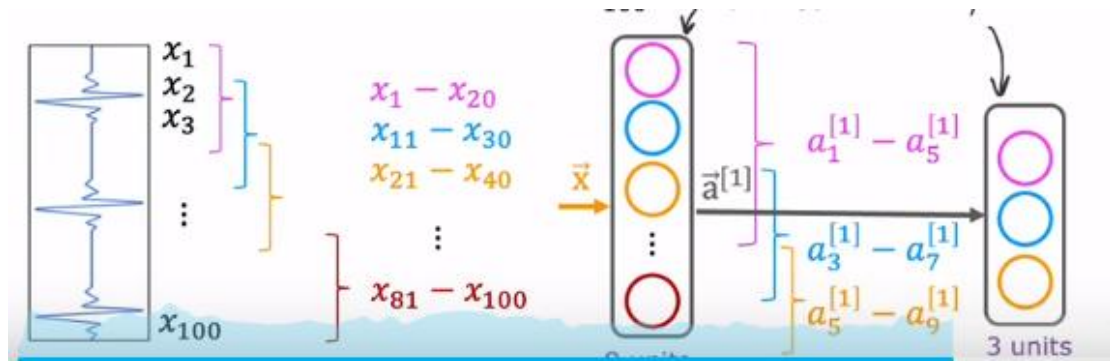
传统的梯度下降由于学习率的限制，如果学习率太小，运行可能太慢。按图中的线路进行梯度下降，显然可以适当提高学习率。对此，可以使用 Adam 算法：如果发现学习率太小，就一步一步提高 alpha。同样，Adam 也可以减小学习率。

如果一个参数，比如 w 或 b ，一直来回震荡，那么 Adam 会稍微减小 alpha。如果在同一方向保持增长或下降，Adam 则会提高学习率。这样，梯度下降的每一小步，alpha 都在调整 adam 通常比梯度下降快很多

其他的网络层类型：

神经网络中，第二个隐藏层的神经元来自前一层每个激活值的函数，但是某些情况下可能会选择使用不同类型的层

另一种层的类型为卷积层。比如输入一个图像，卷积层中一个神经元无法查看整个图像，而是只查看其中一些像素（只关注一小片区域），可以提高运算速度，对训练数据数量的要求较低，不容易过拟合



上面的神经网络有两个卷积层。输入一个时间序列图像，第一个卷积层读取各区域的数据，不同的神经元激活后传第二个卷积层，其神经元读取前一层部分神经元的激活数据并激活，后面可以接一个 sigmoid 层输出二分类结果。这样就构建了卷积层和卷积神经网络

神经网络反向传播：

调整网络中的权重和偏置，使得网络的输出更加接近实际目标值。

• 前向传播：

将输入数据通过网络进行传递，从输入层经过隐藏层到达输出层。在每一层，计算激活值，通常使用激活函数对加权和进行非线性变换。

• 计算损失：

使用损失函数计算网络输出与实际目标之间的误差（损失值）。

• 反向传播：

计算梯度：根据损失函数对每个权重的偏导数，计算梯度。这是通过链式法则来完成的，从输出层开始，逐层向后计算梯度。

更新权重：利用梯度下降算法（或其变种，如 Adam 优化器）更新网络中的权重和偏置，以减少损失函数的值。

重复训练：

将训练数据分批输入网络，重复执行前向传播、损失计算和反向传播步骤，直到网络的性能（通常是通过验证集来评估）达到预期水平或收敛。

模型评估:

把一定比例, 比如 70% 的数据划分为训练集用于训练模型, 剩下的作为测试集评估模型
训练数据的成本函数 J_{train} 比较小, 但是测试集的 J_{test} 比较大, 模型效果不好

Fit parameters by minimizing $J(\bar{w}, b)$ to find $\bar{w}, b$

E.g.,

$$J(\bar{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\bar{w}, b}(\bar{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\bar{w}, b}(\bar{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Compute test error:

$$J_{test}(\bar{w}, b) = -\frac{1}{m_{test}} \sum_{i=1}^{m_{test}} \left[y_{test}^{(i)} \log(f_{\bar{w}, b}(\bar{x}_{test}^{(i)})) + (1 - y_{test}^{(i)}) \log(1 - f_{\bar{w}, b}(\bar{x}_{test}^{(i)})) \right]$$

Compute train error:

$$J_{train}(\bar{w}, b) = -\frac{1}{m_{train}} \sum_{i=1}^{m_{train}} \left[y_{train}^{(i)} \log(f_{\bar{w}, b}(\bar{x}_{train}^{(i)})) + (1 - y_{train}^{(i)}) \log(1 - f_{\bar{w}, b}(\bar{x}_{train}^{(i)})) \right]$$

把这种方法用于分类模型可以评估模型在训练集和测试集的表现

J_{train} 在实际中可能远低于泛化的误差, 也就是新示例中的平均误差, 因为可能过训练

模型选择&交叉验证测试集的训练方法:

把数据按比例分成训练集、交叉验证集和测试集, 比如 60%, 20%, 20%

size	price	
2104	400	training set 60%
1600	330	
2400	369	
1416	232	
3000	540	
1985	300	
1534	315	cross validation 20%
1427	199	
1380	212	test set 20%
1494	243	

$(x^{(1)}, y^{(1)})$
 $\vdots$
 $(x^{(m_{train})}, y^{(m_{train})})$
 $m_{train} = 6$

$(x_{cv}^{(1)}, y_{cv}^{(1)})$
 $\vdots$
 $(x_{cv}^{(m_{cv})}, y_{cv}^{(m_{cv})})$
 $m_{cv} = 2$

$(x_{test}^{(1)}, y_{test}^{(1)})$
 $\vdots$
 $(x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$
 $m_{test} = 2$

交叉验证集是一个额外的数据集, 用于检查或信任检查不同模型的有效性或真实性, 有时也叫做验证集或开发集

Training error: $J_{train}(\bar{w}, b) = \frac{1}{2m_{train}} \left[\sum_{i=1}^{m_{train}} (f_{\bar{w}, b}(\bar{x}^{(i)}) - y^{(i)})^2 \right]$

Cross validation error: $J_{cv}(\bar{w}, b) = \frac{1}{2m_{cv}} \left[\sum_{i=1}^{m_{cv}} (f_{\bar{w}, b}(\bar{x}_{cv}^{(i)}) - y_{cv}^{(i)})^2 \right]$ (validation error, dev error)

Test error: $J_{test}(\bar{w}, b) = \frac{1}{2m_{test}} \left[\sum_{i=1}^{m_{test}} (f_{\bar{w}, b}(\bar{x}_{test}^{(i)}) - y_{test}^{(i)})^2 \right]$

分别计算三个数据集的 J 。其中 J_{cv} 为交叉验证错误, 有时也叫验证错误、开发错误

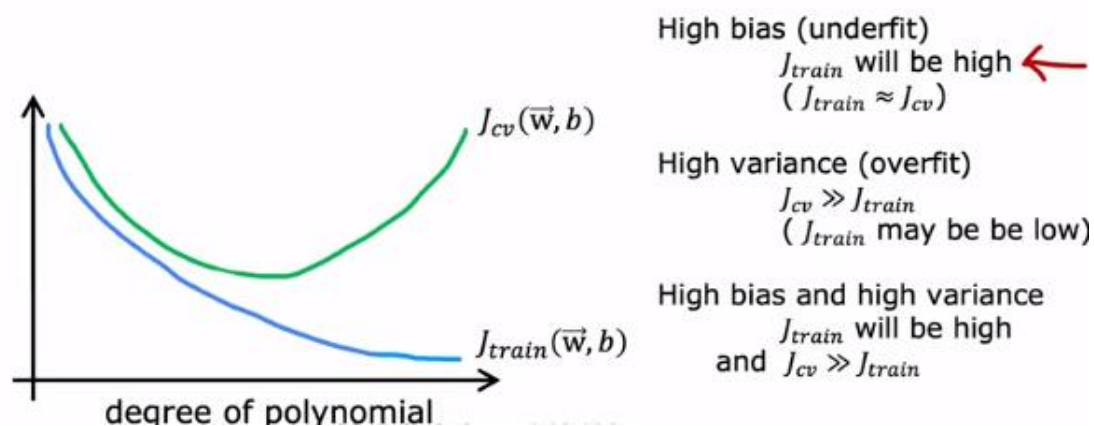
比如对数据进行多项式拟合：先用训练集训练模型，从一阶到十阶，分别算出交叉验证集的 J_{cv} ，找到最小的 J_{cv} 确定阶数，再用测试集的 J_{test} 评估

用到神经网络中：如果要确定合适的隐藏层层数与神经元个数，根据不同情况利用训练集建立模型，然后用 J_{cv} 分别进行评估，最后用 J_{test} 报告泛化误差

问题：将数据划分为训练集、交叉验证集和测试集，交叉验证集是不是多余了？用训练集训练多个模型后，分别用交叉验证集进行验证，确定合适的建模方法，最后用测试集评估其泛化误差。只划分为训练集和测试集，然后用测试集替代交叉验证集进行验证，这样不行吗？如果直接用测试集来选择模型（比如选择超参数或模型结构），就等于在模型选择过程中“看到了”测试集的数据。这样会导致模型在测试集上的表现被“优化”过度，失去了测试集应有的“新数据”评估作用，进而出现过拟合、模型选择不公正等问题。（说白了就是没有交叉验证集的话，训练的模型只能确定是最适合测试集的，碰到新数据可能泛化能力不行）

通过偏差与方法进行诊断：

高偏差意味着欠拟合，高方差是过拟合。欠拟合时 J_{train} 和 J_{cv} 都很高，过拟合时 J_{train} 比较低，但 J_{cv} 可能很高。拟合恰当时，没有高偏差或高方差

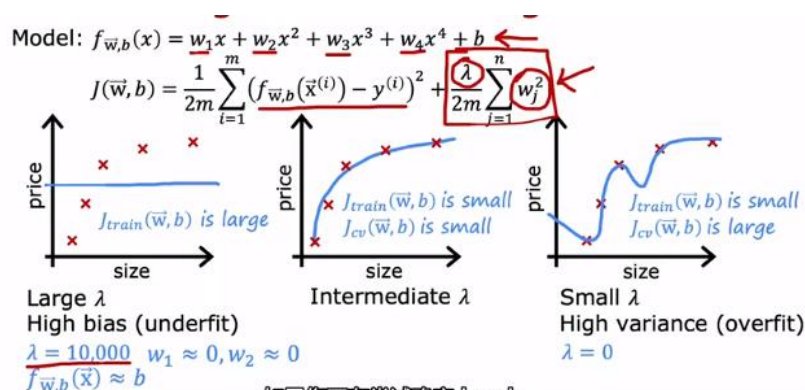


多项式拟合时，随着阶数提高， J_{train} 会逐渐变小，但后面会出现过拟合； J_{cv} 先减后增， J_{cv} 最小时，这时的阶数最适合进行拟合

阶数过高时，对训练集进行了过拟合，到了验证集，可能同时出现高偏差和高方差。高偏差甚至在训练集表现不佳，而高方差是在交叉验证集和训练集的表现差距很大

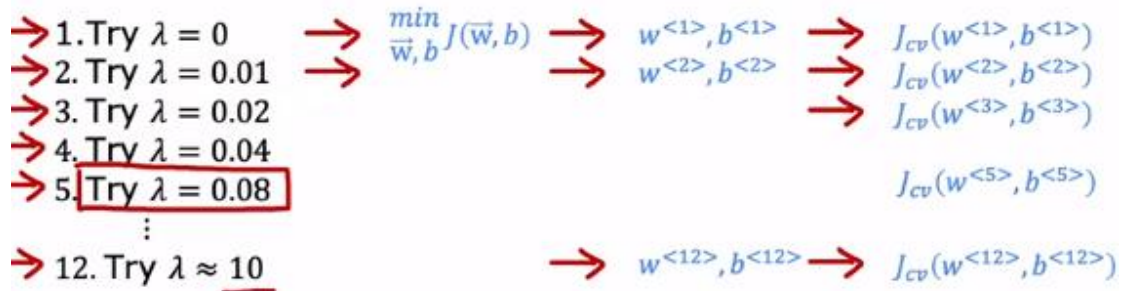
正则化、偏差、方差：

通过正则化方法拟合（引入惩罚函数处理 w_1 等参数）时：



需要选择一个合适的 lambda，可以从 0 到 10 逐个尝试，计算交叉验证集的 J_{cv} ，选择正则化参数最佳值，最后用 J_{test} 评估泛化误差

$$\text{Model: } f_{\bar{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$$

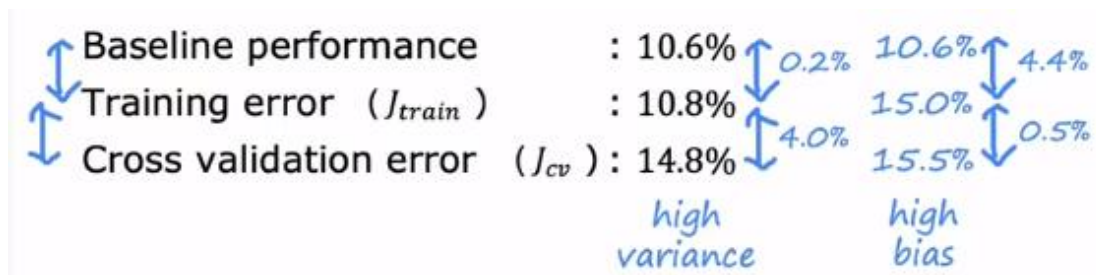


Pick $w^{<5>}, b^{<5>}$

Report test error: $J_{test}(w^{<5>}, b^{<5>})$

lambda 越大， J_{train} 越大， J_{cv} 先减后增，选择 J_{cv} 最小时的 lambda

制定一个用于性能评估的基准：



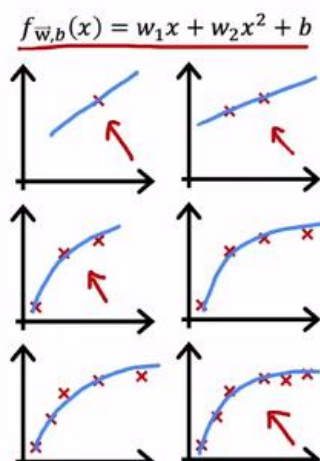
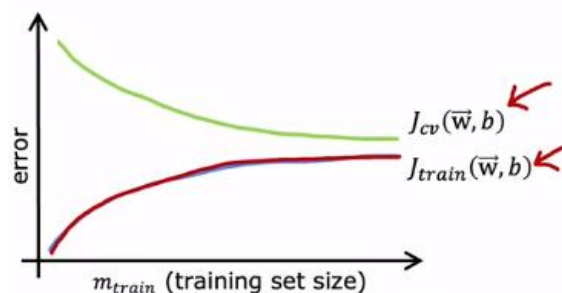
比如，将人类对图像的处理能力作为基准，评估机器的图像处理能力。 J_{train} 与基准相差较小，而 J_{cv} 远大于 J_{train} ，意味着有高方差；如果 J_{train} 与 J_{cv} 相近，但是 J_{train} 远大于基准，意味着有高偏差。如果三个值之间的差距都很大，那么同时具有高偏差和高方差

学习曲线：

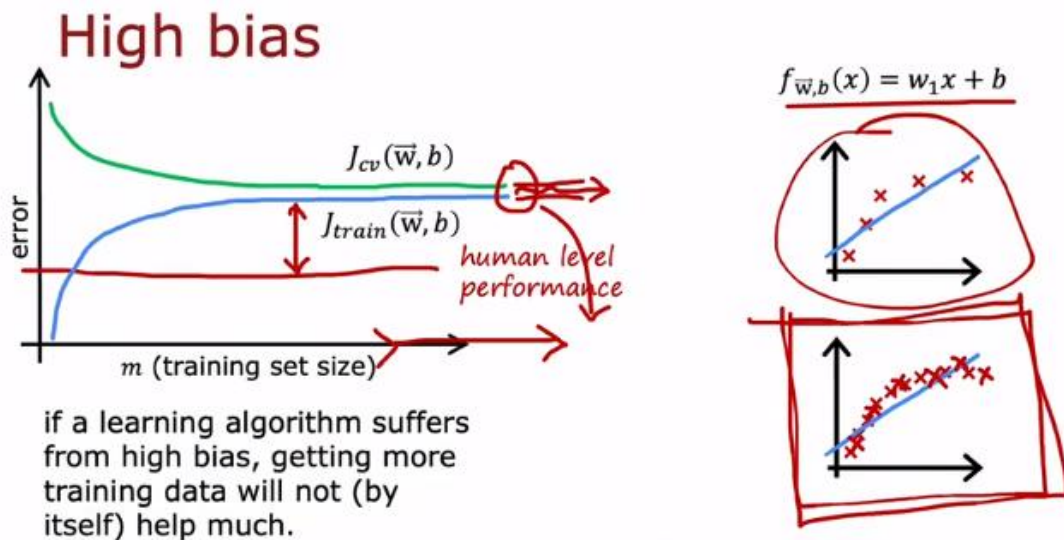
Learning curves

J_{train} = training error

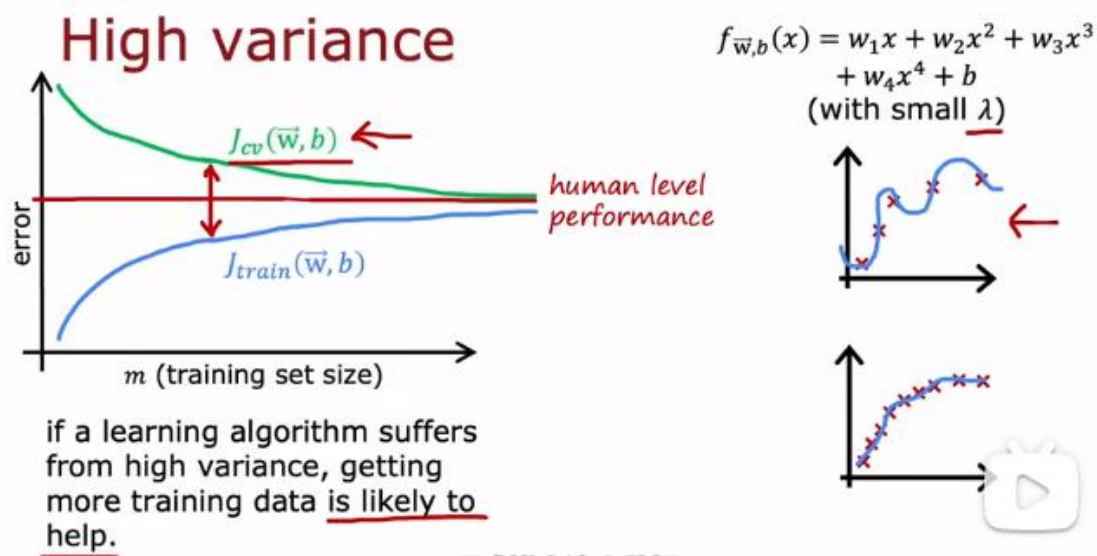
J_{cv} = cross validation error



J_{train} 随着训练集变大而增加, J_{cv} 会减小, 一般情况下 J_{cv} 会大于 J_{train}
 如果算法具有高偏差, 可以使用更多的数据训练, 但数据量足够大时, 增加数据不一定有多大帮助, 可能需要考虑别的方法



在高方差的情况下, J_{cv} 远大于 J_{train} , J_{train} 可能低于人类的基准 J , 因为过度训练
 这时使用更多的数据可能有较大帮助, 可以降低 J_{cv}



针对高方差和高偏差的处理方法:

Debugging a learning algorithm

You've implemented regularized linear regression on housing prices

$$J(\bar{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\bar{w},b}(\bar{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

But it makes unacceptably large errors in predictions. What do you try next?

- | | |
|---|---------------------|
| → Get more training examples | fixes high variance |
| → Try smaller sets of features $x, x^2, x^3, \dots$ | fixes high variance |
| → Try getting additional features | fixes high bias |
| → Try adding polynomial features $(x_1^2, x_2^2, x_1x_2, \text{etc})$ | fixes high bias |
| → Try decreasing λ | fixes high bias |
| → Try increasing λ | fixes high variance |

给与额外的新特征，或引入多项式特征（某些特征相乘、相除、平方等），或减小 λ 可以处理高偏差。 λ 较大时，惩罚项使 w 参数较小，限制模型复杂度从而可能欠拟合，减小后拟合效果提升防止高偏差

方差与偏差：

大型神经网络一般具有较低的偏差

如果在训练集上表现不好，认为是高偏差，可以扩大神经网络规模；如果在交叉验证集上表现不好，认为算法有高方差，尝试获取更多数据，然后重新测试

大型神经网络计算量太大，可以采用 GPU 或图像处理单元进行加速。硬件传统上用于加速计算机图形，但实际上对于加速神经网络也很有用。但即使加速器超过一定程度，神经网络太大，训练时间太长，变得不可行

尝试增加数据来改良神经网络时，适用于你可以访问大量数据的应用程序从而进行训练。有时达到一定程度后无法收集更多数据

神经网络太大时是否会产生高方差问题？一个具有良好正则化的大型神经网络通常比较小的神经网络做得更好。神经网络规模变大可能出现过拟合问题，但事实证明，如果适当对其进行正则化，大型神经网络往往能比较小的好。在正则化适当的前提下，大型神经网络出现的问题一般是训练时间长，而过拟合问题不太明显，不会很大程度上影响程序

机器学习的开发迭代：

选择模型（模型结构、数据等）——建立模型（第一次一般效果都不是非常好）——判断模型的问题（误差、方差、错误分析）——确定解决方法后更改模型

误差分析：

比如交叉验证集有 500 个数据，有 100 个被错误分类。查看这 100 个数据找到错误分类的原因。尝试把它们分组寻找共同的属性或特征或别的：比如将邮件分类为普通邮件和垃圾邮件，发现有些存在拼写错误的邮件被分类为垃圾邮件，那就查看所有拼写错误的邮件，看看是什么导致有些文件被错误分类。

错误分类的邮件有多种类别，某个邮件可能同时属于多个类。

比如有制药相关的邮件被归类为垃圾邮件，可以扩大数据集，或者单独找到更多的制药相关的邮件放入数据集进行训练，从而对算法进行特定更改

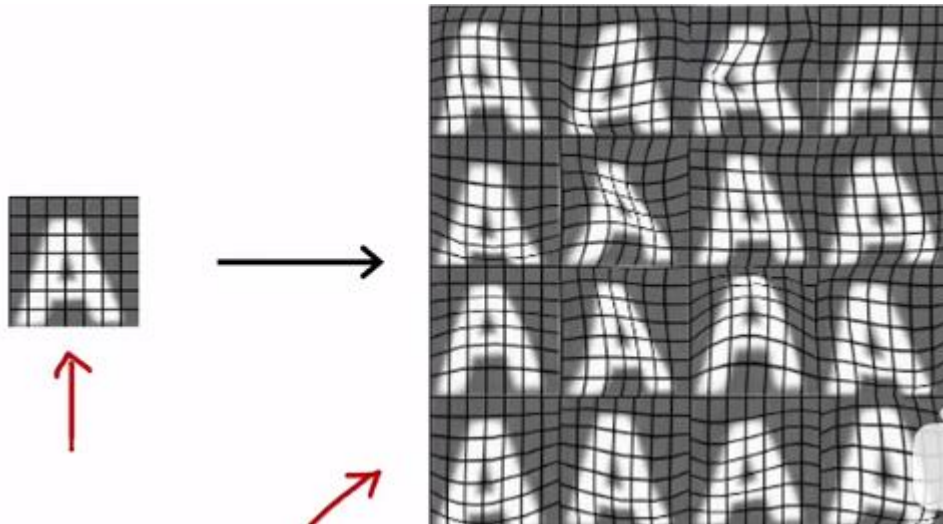
添加更多数据：

尝试获取更多类型的数据可能慢而昂贵

比如在分类问题中，有的数据没有分类标签，除了找更多的数据，还可以把这些缺失的标签补上。又比如通过引入新的特征，比如 x^2 , xy 等

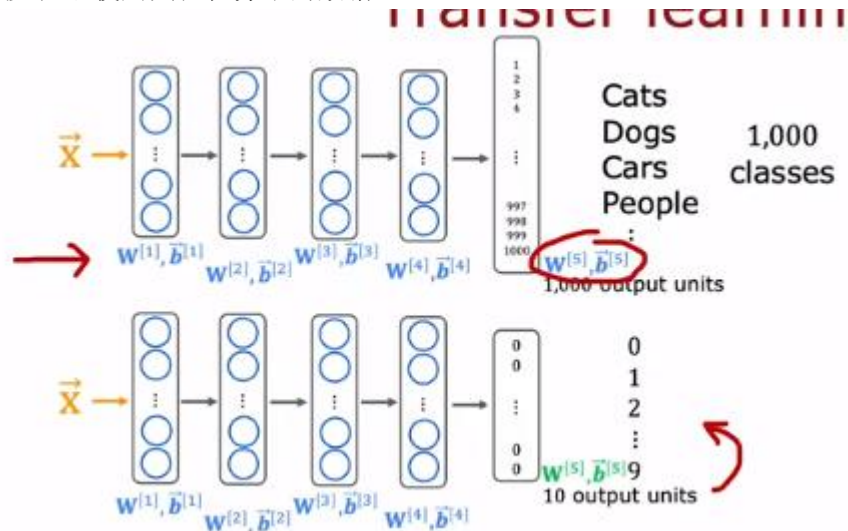
数据增强：用现有的数据示例创建一个新的数据示例。

比如要建立模型识别图像中的字母，有个数据是一个写有“A”的图像，可以对图像放缩、旋转、增强对比度等，得到新的数据，还可以进行网格扭曲



这里引入随机噪声可能没什么用，因为随机修改每个像素的灰度对整个图像的影响较小，因此考虑数据增强的一种方法是如何修改、扭曲或在数据中引入更多噪声
在文本编辑器中使用不同字体什么的也可以生成大量数据
又比如进行语音识别，一个数据是安静环境下人的语音，剪辑，引入背景噪声，如汽车声等，得到新的数据

迁移学习-使用其他任务中的数据：



预训练一个大型神经网络处理数据集 X ，隐藏层有 4 个，得到各层的参数。训练一个新的神经网络，隐藏层的 w, b 参数可以直接使用之前神经网络得到的参数进行初始化，但是改动了输出层， w_5 和 b_5 需要重新计算

之前的大型神经网络学习得到的参数直接传给新的神经网络，新的神经网络在该基础上进行学习。如果新神经网络使用的数据集较小，参数调优后也能有较好效果

实际中可以下载别人开源的神经网络，用它们的神经网络与较大的数据集完成第一步参数计算，微调后用于自己的神经网络，这时我们可以用较小的数据集训练

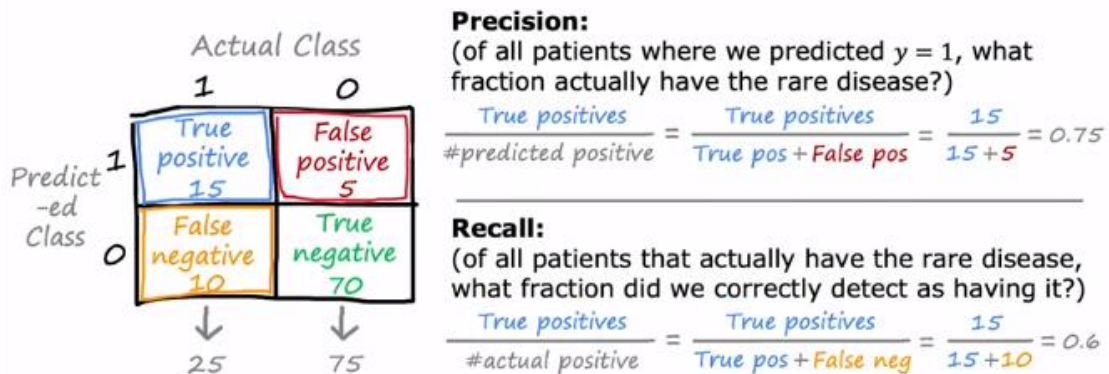
倾斜数据集的误差指标：

如果算法对某一种数据有偏斜，最后可能出现偏差。

比如要通过分类诊断一种发病率 0.5% 的罕见病，最后有 99% 的数据被正确诊断，1% 错误诊断，这样的准确率看起来可以，但是因为是罕见病，大约 0.5% 的人会患病，却有 1% 的人被认为患病，显然结果有较大问题

有三个模型去预测上面的罕见病，准确率分别是 0.995, 0.99, 0.988，其实单凭准确率很难判断哪个模型比较好。比如准确率 0.995 的模型一直判断没患罕见病，导致有的患者可能也被当成了健康人；而准确率相对较低的模型至少在一些情况下可以正确诊断患者
对于偏斜数据集，不能单凭准确率进行度量。可以通过精确率和召回率

$y = 1$ in presence of rare class we want to detect.



同时查看精确率和召回率，都很高的话说明效果好

精确率和召回率的权衡：

还是上面的例子。

精确率是在模型预测为正类的样本中，实际为正类的样本所占的比例，衡量的是模型在预测正类时有多准确；召回率是指在所有实际为正类的样本中，被模型正确预测为正类的比例，衡量的是模型对正类样本的覆盖能力。

Trading off precision and recall

Logistic regression: $0 < f_{\bar{w},b}(\vec{x}) < 1$

→ Predict 1 if $f_{\bar{w},b}(\vec{x}) \geq \cancel{0.5} 0.7$

→ Predict 0 if $f_{\bar{w},b}(\vec{x}) < \cancel{0.5} 0.7$

Suppose we want to predict $y = 1$ (rare disease) only if very confident.
→ higher precision,

→ $\text{precision} = \frac{\text{true positives}}{\text{total predicted positive}}$

→ $\text{recall} = \frac{\text{true positives}}{\text{total actual positive}}$

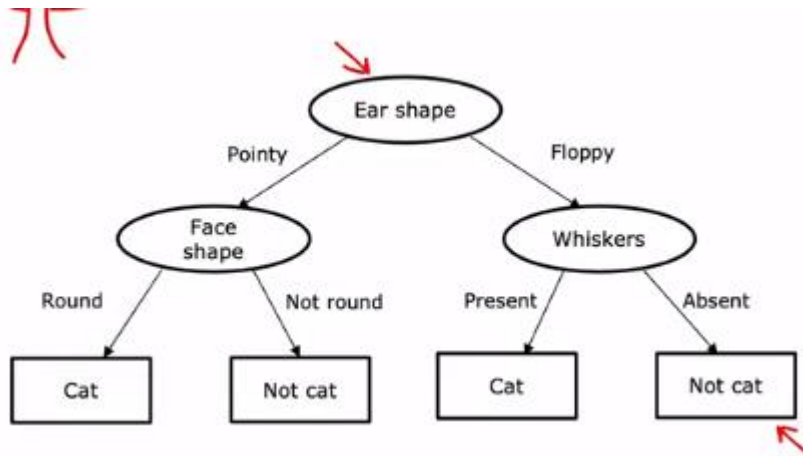
修改二分类的阈值可以提高模型性能。比如把阈值从 0.5 改成 0.7，这时预测为 1 的概率变小，精确率会变高，但是召回率会降低。这种方法只有在相当自信的条件下才会将之预测为 1 类。或者把阈值降低到 0.3，影响与之相反，提高召回率有助于找到更多可能患病的人
可以手动选择阈值以衡量精确率和召回率，也可以利用 F1 评分进行权衡

$F1 = 2 / (1/\text{Precision} + 1/\text{Recall}) = 2 * P * R / (P + R)$ 。看哪个阈值 F1 评分高就选哪个

F1 评分比单纯计算精确率和召回率的平均值更好。比如精确率、召回率分别是 0.02 和 1，取平均是 0.501，看起来可能相对不错，但 F1 评分取调和平均值，更关注其中较低的值，这时的 F1 评分很低，能确定模型效果不好

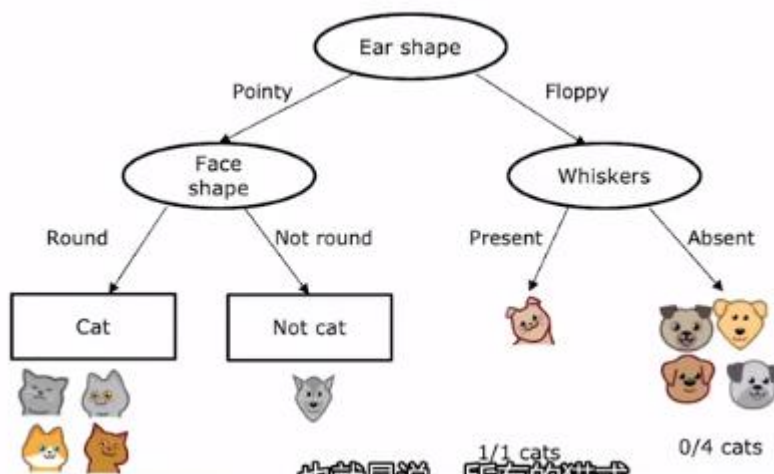
决策树模型：

比如，输入多个特征，分类判断一个动物是不是猫：



输入一个新的示例，从上面决策树的顶端节点开始，也就是树的根节点，这个节点的特征是耳朵的形状。如果耳朵是尖的，那就沿树的左侧分支往下走到达第二个节点：脸的形状，再次判断，这时分类得到结果。上面那些椭圆形是分类节点，矩形表示分类结果

学习过程：

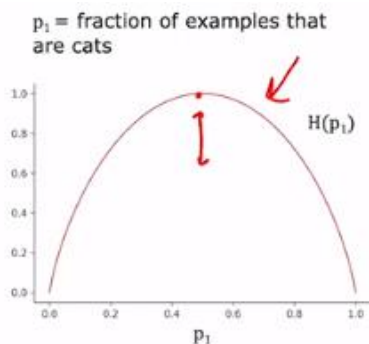
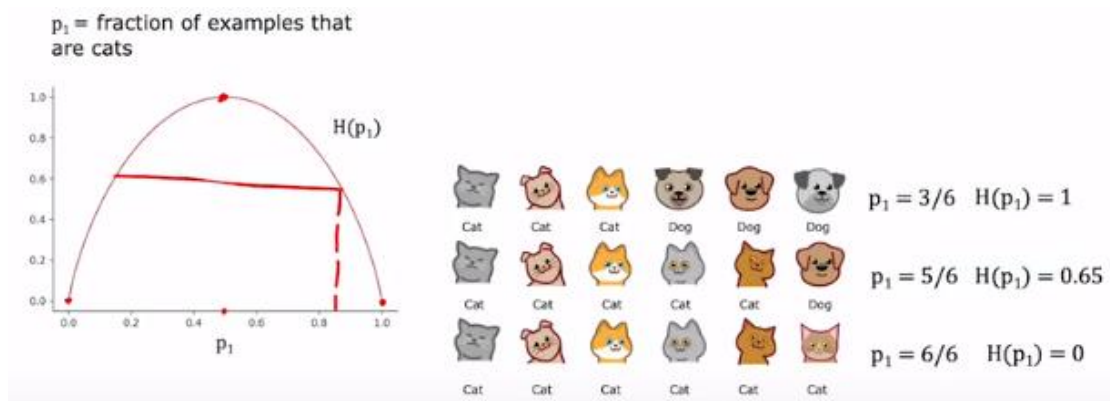


在决策树的根节点和左右分支，必须决定要不要让他们分开，用什么特征分开，提高纯度
什么时候停止分裂：当一个节点全是一个类；或者达到树的最大深度（最大深度确保决策树不会太大）；或继续分裂对纯度的提升小于一个阈值（防止过拟合）；或某个节点的示例数小于一个阈值

纯度：

熵是衡量一组数据纯度的指标

比如有一组数据，里面有三只猫三只狗， p_1 是猫的比例 0.5，熵记作 H ，在这里是关于 p_1 的函数。如果 p_1 是 0.5，这时熵最大，是 1，表示不纯；如果是 0 或 1，熵就是 0，纯度高



$$p_0 = 1 - p_1$$

$$H(p_1) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

$$= -p_1 \log_2(p_1) - (1 - p_1) \log_2(1 - p_1)$$

Note: " $0 \log(0)$ " = 0

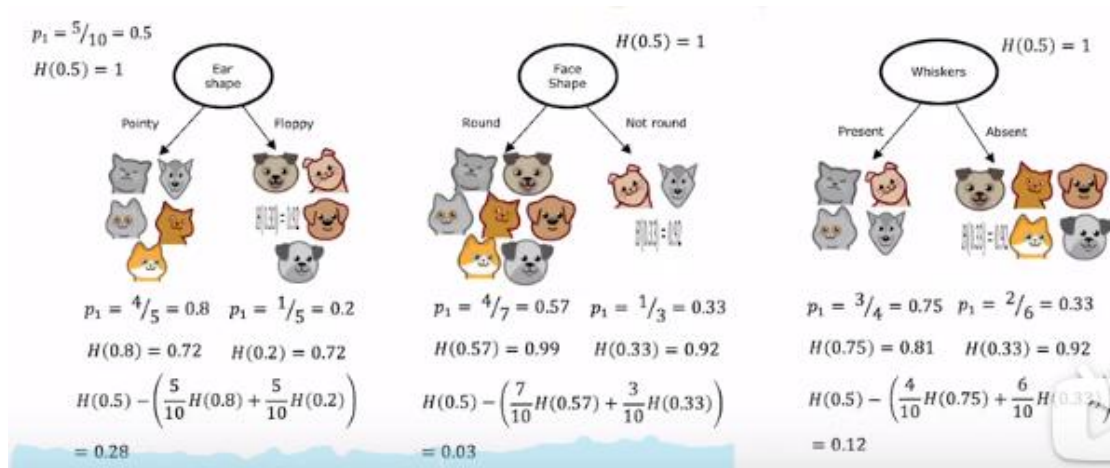
$H(p_1)$ 中, $\log$ 的底数是 2, 如果是别的数字, 比如 e , 相当于把函数进行缩放, 但是这样不方便解释结果, 因为熵达到最大时不是一个很好的整数。为了方便解释, 取 p_1 为 0 或 1 时熵为 0

选择拆分信息增益:

选择什么特征可以减少熵或减少杂质, 或最大化纯度?

决策树学习中, 熵的减少称为信息增益

在根节点, 对于相同的示例, 使用不同特征进行拆分, 会得到不同的分类结果和熵



与其比较这些熵, 不如对它们加权平均。将左右分支的熵加权平均, 比如按分到两个分支的正类在该节点的比例确定权值, 比一下结果。这里的 0.28 等结果就是信息增益, 表示进入节点后熵的减小量

熵减比较小, 低于一个阈值的, 可以不再拆分, 因为不能再明显提高纯度。别的可以拆

信息增益公式：

Information gain

$$= H(p_1^{\text{root}}) - \left(w^{\text{left}} H(p_1^{\text{left}}) + w^{\text{right}} H(p_1^{\text{right}}) \right)$$

整合：

在决策树的各个地方使用信息增益。从根节点开始，计算所有特征的信息增益并选择要分割的特征（信息增益最大的那个）。根据选择的特征将数据集划分，并创造树的左右分支。重复分割操作，直到满足停止条件（看前面“学习过程”）

决策树使用了递归算法。数据集分割后，在分割后的数据集基础上创建新的分支

独热编码：

如果特征可以采用两个以上的离散值，可以用 one-hot 编码。（给决策树创建多个分支也行，但是独热编码可能更合适）。

在一群动物里面识别猫，根据耳朵形状这一特征使用独热编码：这样就构建了三个特征，每个特征都只能取 0 或 1，而不是一个特征能取三个可能的值：

	Ear-shape	Pointy ears	Floppy ears	Oval ears	Face shape	Whiskers	Cat
	Pointy	1	0	0	Round	Present	1
	Oval	0	0	1	Not round	Present	1
	Oval	0	0	1	Round	Absent	0
	Pointy	1	0	0	Not round	Present	0
	Oval	0	0	1	Round	Present	1
	Pointy	1	0	0	Round	Absent	1
	Floppy	0	1	0	Not round	Absent	0
	Oval	0	0	1	Round	Absent	1
	Floppy	0	1	0	Round	Absent	0
	Floppy	0	1	0	Round	Absent	0

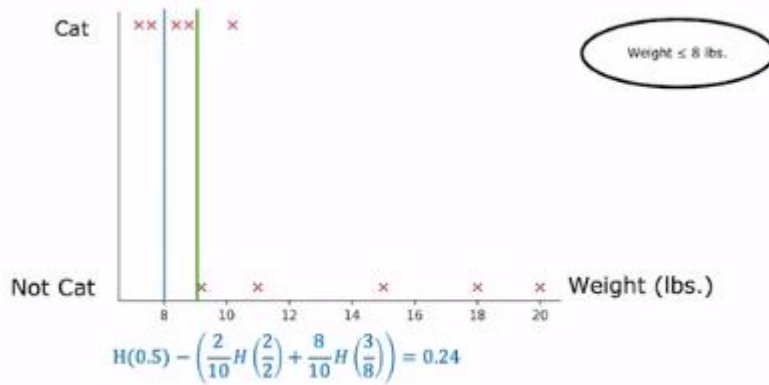
所构建的三个特征中，肯定有一个特征是 1，得名独热编码。对这些特征使用决策树就行其他特征也一样，用 1 代表存在，0 代表不存在，对分类特征进行编码以便输入神经网络

	Pointy ears	Floppy ears	Round ears	Face shape	Whiskers	Cat
	1	0	0	Round 1	Present 1	1
	0	0	1	Not-round 0	Present 1	1
	0	0	1	Round 1	Absent 0	0

连续有价值的功能：

之前的决策树使用特征是离散值，实际上也可以使用连续的、有价值的数据

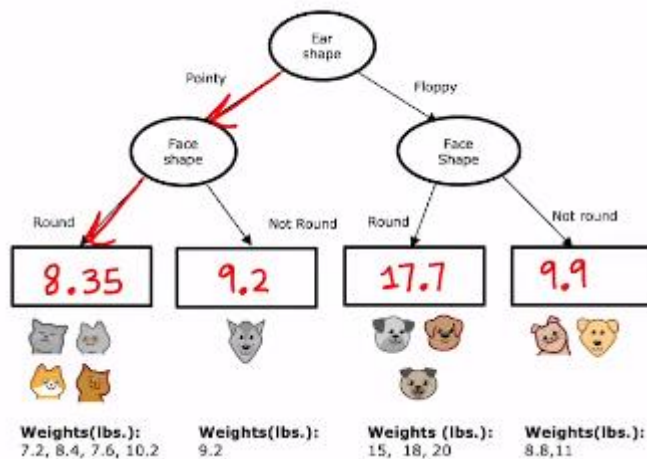
在之前特征的基础上引入重量这一特征用于识别猫，决策树的运行方法与之前类似决定如何分割权重特征：



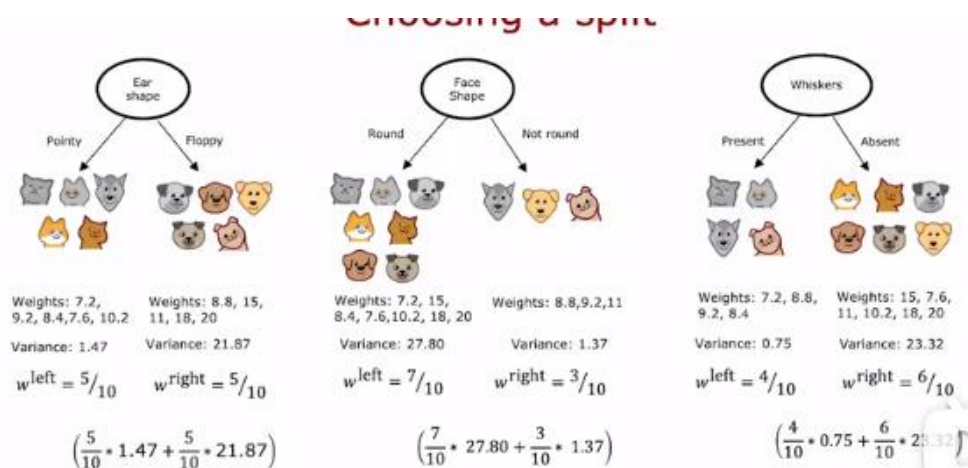
比如，如果要设置权重是 8，以重量为横轴，分类标签为纵轴，在 weight 是 8 的地方画一条线，这时候线左边有 2 个猫，右边有 3 个猫 5 个狗，然后就能计算信息增益
尝试 x 轴的多个值，排序，选择能提供最高信息增益的那个，可以选这个进行分割

回归树：

决策树也可以用来回归预测。比如，利用之前那些特征，把重量作为输出
当到达一个节点，决策树会根据这里权重的平均值进行预测



根据耳朵形状进行决策得到两个分支，再根据脸型进行决策得到分支，以及相应的权重，再根据胡须进行分裂。构建决策树时不是试图减小熵，那是对杂质的度量，而是通过减小 Y 的权重



选择拆分的一个好方法是根据这些权重的加权方差值，算出信息增益，选择能最大程度减少方差的特征作为分割特征

对于输入的新数据点，从根节点开始往下走直到到达一个叶节点。通过递归将数据分割成越来越小的子集，逐渐逼近目标变量实际值，最终在叶节点处返回该数据点的预测值

使用多个决策树：

使用单个决策树的缺点是，可能对数据的微小变化高度敏感，可以构建很多树，称为树集合。改变一个训练样本，要分割的特征可能不同，然后就有了两个不同的决策树。

可以训练多个决策树，投票进行最终预测，得票多的预测结果作为最终分类结果。

可以通过有放回抽样的方式选取训练集：从数据集抽取多个随机训练集（训练集之间必须互不相同，训练集内部如果有数据相同的话没事），分别训练决策树。

随机森林：

利用不同的训练集训练多个决策树得到树集合，最后投票。

因为上面的方法在实际上，可能每个节点选择的特征会相似，对此可以用另一方法，在每个节点，选择要分裂的特征时，如果有 n 个特征可选，选择一个随机的包含 k 个特征的集合， $k < n$ ，让决策树只能从这 k 个特征中选择。 n 很大时，一种经典的方法是令 $k = \sqrt{n}$ 。

XGBoost：

XGBoost 运行速度快，开源实现易于使用。

先训练一个决策树，查看做的不好的地方，以此训练新的决策树。因此，**XGBoost** 不是查看所有训练示例，而是将更多注意力集中在表现不好的示例子集上并获得新的决策树，使下一个决策树在这里做得更好，也就是“刻意练习”。

先用所有训练示例训练决策树（不是随机森林的随机训练集），找到错误，训练新的决策树，但是每次从示例中挑选一个例子时，有更高的可能从分类错误的例子中挑选一个。多次迭代。

XGBoost 还内置了正则化以防止过拟合。

XGBoost 实际上为不同的训练示例分配了不同的方法，所以它实际上不需要生成很多随机选择的训练集，采样更有效。

XGBoost 除了用于分类，也可以用于回归，方法类似。

何时使用决策树：

决策树和树集成通常用于表格数据，也称为结构化数据。如果数据集看起来像一个巨大的电子表格，可以考虑决策树。比如通过面积等特征预测房价。

不建议在非结构化数据中使用决策树，比如图像、视频、音频和文本，这些不太可能用电子表格存储。

决策树和树集成的优势是训练速度比较快，而且小型决策树方便解释。

与之相比，神经网络适用于所有类型数据，从电子表格到音频等，以及包含结构化和非结构化数据的混合数据，但是神经网络可能比决策树训练时间长，不过可以进行迁移学习来解决问题。此外，可以将多个神经网络串联，构建更大的机器学习系统。

如果要构建多个机器学习模型协同工作的系统，串联多个神经网络可能更容易，用梯度下降。

将它们一起训练

聚类：

一种无监督学习算法。

监督学习中，输入数据时会输入对应的分类标签，聚类算法不会输入标签

K-means 聚类：

K-means 是最常见的聚类算法。第一步猜测哪里可能是集群的中心（假设设置要查找两个集群），簇的中心称为簇质心，中心标记为红十字和蓝十字。遍历每一个数据点，看它是否更接近其中一个十字，把每个点分配给离它最近的集群质心，红、蓝十字分配到的点变成红色和蓝色。把红色簇的质心从原来红十字移动到所有红点的质心，蓝色也是，得到新的集群质心位置。再次遍历每个示例，重复这个过程。簇的质心不再发生变化时结束。

优化目标：

K-means optimization objective

$c^{(i)}$ = index of cluster $(1, 2, \dots, K)$ to which example $x^{(i)}$ is currently assigned

μ_k = cluster centroid k

$\mu_{c^{(i)}}$ = cluster centroid of cluster to which example $x^{(i)}$ has been assigned

Cost function

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

$\min_{c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K} J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K)$

目标是使得平方距离最小化。这里的成本函数 J 也叫失真成本函数

不断将数据点分配到最近的簇质心，尝试最小化 J。J 保持不变时，说明 K-means 已经收敛

初始化 K-means：

选择簇的数量 k ， k 必须小于总示例数 m 。随机选择 k 个训练样例（只选一部分数据，不是 m 个），这是一个训练集，通过这些数据，运行 K-means 以设置初始的集群质心。计算成本函数 J，多次迭代后找到 J 最小的那个

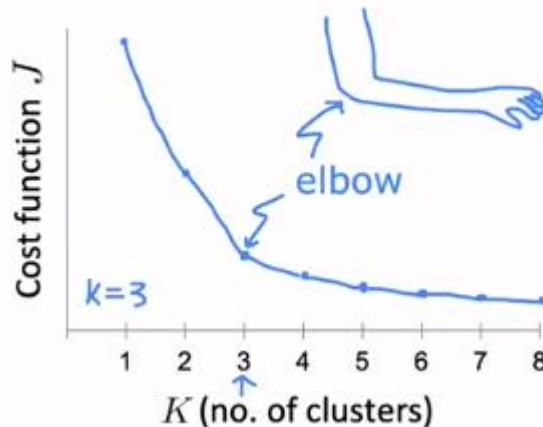
这是随机初始化，计算简单，但是可能收敛到局部最优解，尤其是数据分布不均匀时所以可以进行多次随机初始化，多次迭代计算，找到效果最好的

选择聚类数量：

聚类是无监督学习算法，没有明确的指标可以确定到底有几个簇

选择 k 值的方法：

一种方法是肘法，也就是画出成本函数 J 关于 k 的函数，计算不同 k 情况下的成本函数，画出曲线，找到肘部（曲线总体是下降的，肘部右侧下降速度明显变慢，下图中肘部是 3）



up 一般不用这个方法。通过肘法判断集群数量，很多时候集群数量其实是模棱两可的
直接选择 J 最小时的 k 是不行的，这会导致我们总是选 k 的最大可能值，而 k 越多 J 就越小
另一种方法是看聚类结果在下游的表现。在不同 k 时运行 K-means，看聚类效果，以及在实际中是否符合我们的需要，能否解释

发现异常事件：

对于数据 X_{test} ，计算它出现的概率，小于一个阈值则不容易发生，发生时可能是异常情况

高斯正态分布：

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

对符合正态分布的数据，用这个可以找异常值。高中讲过不写了

异常检测算法：

有 x_1, x_2 等特征，进行一个密度估计，计算任意特征向量的概率。假设各特征之间互相独立（有点相关性也不影响）， $p(x) = p(x_1; \mu_1, \sigma_1^2) \cdot \dots \cdot p(x_n; \mu_n, \sigma_n^2)$ ，

1. Choose n features x_i that you think might be indicative of anomalous examples.
2. Fit parameters $\mu_1, \dots, \mu_n, \sigma_1^2, \dots, \sigma_n^2$

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)} \quad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2$$

3. Given new example x , compute $p(x)$:

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi}\sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right)$$

对数据计算 $p(x)$ ，小于一个阈值，比如 0.02，就有异常

开发与评估异常检测系统：

对已有的异常检测算法，尝试修改某些参数，看看能不能提高效果

可以训练分类模型，根据以往检测到的异常和正常的数据打上分类标签，划分数据集然后训练。如果异常数据很少，用分类模型进行检测时，可以只划分为训练集和交叉验证集，不设置测试集。把所有异常数据都放到交叉验证集中，训练集没有异常数据。调整分类的阈值，使模型效果尽可能好。缺点是因为没有测试集，很难判断实际效果。如果异常值比较多，那还是用之前的方法

异常检测与监督学习对比：

正例数量比较少时，异常检测算法可能更好。如果异常值出现的方式有很多且正例难以覆盖所有异常情况，可以进行异常检测算法

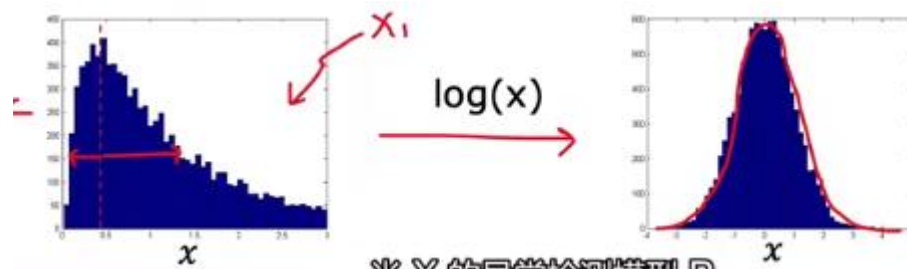
正例足够，有足够种类和数量的异常数据用于训练，可以监督学习

选择使用什么特征：

监督学习中，如果没有完全正确的特征，或者有一些与问题无关的额外特征，其实也是可以的，因为算法必须监督有足够标签的信号。

但是异常检测中，对于运行或仅从未标记数据中学习的异常检测，更难找出要忽略的特征，所以选特征在异常检测中很重要

让数据尽量呈现高斯分布，对于不是高斯分布的数据，通过处理让它接近高斯分布



对不同的特征 x ，尝试使用对数、幂运算等让它更高斯

异常检测过程中，还可以执行错误分析过程以进行异常检测

使用每个特征：

要建立一个推荐系统。假如有多个特征的数据集，4 个用户对 5 个电影进行评价，同时还拥有电影的两个特征。如果评价时有数据残缺，可以用预测的方法把缺失值补上：比如多元线性回归，引入成本函数 J 和正则化项，最小化成本函数得到参数

To learn parameters $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, w^{(n_u)}, b^{(n_u)}$ for all users :

$$J\left(\begin{matrix} w^{(1)}, \dots, w^{(n_u)} \\ b^{(1)}, \dots, b^{(n_u)} \end{matrix}\right) = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

方法类似于线性回归

协同过滤算法:

一种推荐系统算法, 根据用户的相关数据推测用户可能喜欢哪些

Collaborative filtering

Cost function to learn $w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}$:

$$\min_{w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}} \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

Cost function to learn $x^{(1)}, \dots, x^{(n_m)}$:

$$\min_{x^{(1)}, \dots, x^{(n_m)}} \frac{1}{2} \sum_{i=1}^{n_m} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

Put them together:

$$\min_{\substack{w^{(1)}, \dots, w^{(n_u)} \\ b^{(1)}, \dots, b^{(n_u)} \\ x^{(1)}, \dots, x^{(n_m)}}} J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

	Alice	Bob	Carol
Movie1	5	5	?
Movie2	?	2	3

执行梯度下降时, 要同时更新 w, b, x 。这里 x 也是一个参数

$$w_i^{(j)} = w_i^{(j)} - \alpha \frac{\partial}{\partial w_i^{(j)}} J(w, b, x)$$

$$b^{(j)} = b^{(j)} - \alpha \frac{\partial}{\partial b^{(j)}} J(w, b, x)$$

$$x_k^{(i)} = x_k^{(i)} - \alpha \frac{\partial}{\partial x_k^{(i)}} J(w, b, x)$$

比如要评价一部新的电影, 根据用户数据算出 w 等参数, 然后就可以预测这部电影的评分

二进制标签:

➤ Previously:

➤ Predict $y^{(i,j)}$ as $w^{(j)} \cdot x^{(i)} + b^{(j)}$

➤ For binary labels:

Predict that the probability of $y^{(i,j)} = 1$ is given by $g(w^{(j)} \cdot x^{(i)} + b^{(j)})$

$$\text{where } g(z) = \frac{1}{1+e^{-z}}$$

如果要预测的对象结果是一个二进制标签, 比如 0 和 1, 经过回归运算后, 使用 sigmoid 函数转换为二进制标签, 与分类模型类似

Previous cost function:

$$\frac{1}{2} \sum_{(i,j):r(i,j)=1} (\underbrace{w^{(j)} \cdot x^{(i)} + b^{(j)}}_{f(x)}) - y^{(i,j)} \Big)^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

Loss for binary labels $y^{(i,j)}$: $f_{(w,b,x)}(x) = g(w^{(j)} \cdot x^{(i)} + b^{(j)})$

$$L(f_{(w,b,x)}(x), y^{(i,j)}) = -y^{(i,j)} \log(f_{(w,b,x)}(x)) - (1 - y^{(i,j)}) \log(1 - f_{(w,b,x)}(x))$$

Loss for single example

$$J(w, b, x) = \sum_{(i,j):r(i,j)=1} L(f_{(w,b,x)}(x), y^{(i,j)})$$

最终的损失函数与逻辑回归类似，成本函数 J 就是各个 L 求和

均值归一化:

为了让算法运行更快，可以进行归一化，比如均值归一化

现在，4 个人对一部电影打分，有 5 项指标。如果有数据残缺也没事。左边的矩阵，每行表示一个指标，? 表示残缺，每一列是一个人对五项指标的打分。

Mean Normalization

$$\begin{matrix} \rightarrow & \begin{bmatrix} 5 & 5 & 0 & 0 & ? \\ 5 & ? & ? & 0 & ? \\ ? & 4 & 0 & ? & ? \\ 0 & 0 & 5 & 4 & ? \\ 0 & 0 & 5 & 0 & ? \end{bmatrix} & \begin{matrix} 2.5 \\ 2.5 \\ 2 \\ 2.25 \\ 1.25 \end{matrix} \end{matrix} \quad \mu = \begin{bmatrix} 2.5 \\ 2.5 \\ 2 \\ 2.25 \\ 1.25 \end{bmatrix} \quad \begin{bmatrix} 2.5 & 2.5 & -2.5 & -2.5 & ? \\ 2.5 & ? & ? & -2.5 & ? \\ ? & 2 & -2 & ? & ? \\ -2.25 & -2.25 & 2.75 & 1.75 & ? \\ -1.25 & -1.25 & 3.75 & -1.25 & ? \end{bmatrix}$$

先对每个指标的所有打分求平均，得到列向量 μ 。从每个评级中减去它给出的平均评级，比如矩阵第一行第二列的 5，减去对应 μ 第一行的 2.5，得到 2.5，放到对应的位置

用最右边的矩阵学习 w, b, x 参数。因为第五个人没有打分（第五列都是问号），这个算法的效果会使其初始猜测刚好等于其他用户打分的平均值，这比直接全部按 0 处理更合理

通过均值归一，把每一行的平均值变成 0，算法会运行更快一点，而且对于那些没有评价电影或评价数量很少的用户表现得更好，预测更合理

这里也可以对列进行标准化，但是有人完全没有评分，所以规范化按行标准化

寻找相关特征:

比如要推荐一个电影。给定特征 x_1, x_2, x_3 ，这些特征统称为一定要传达一些关于那部电影的内容。给定项目 i 的特征，要找到相关的其他电影，可以尝试找到项目 k ，与特征 i 类似。

可以给定一个特征向量，求它们的平方距离

平方距离最小时，这些特征与给定特征最相关

The features $x^{(i)}$ of item i are quite hard to interpret.

To find other items related to it,

find item k with $x^{(k)}$ similar to $x^{(i)}$

i.e. with smallest distance

$$\sum_{l=1}^n (x_l^{(k)} - x_l^{(i)})^2$$

$$\|x^{(k)} - x^{(i)}\|^2$$

如果只是找特征间的相似性或差异性，可以用平方距离；如果要深入分析特征间的关系等，相关性分析等方法可能更合适

协同过滤与基于内容过滤对比：

协同过滤一般的方法是基于与你相似的用户的评分推荐新商品，基于内容的过滤会根据用户的特点进行推荐，通过比对特征确定哪些物品与你更匹配

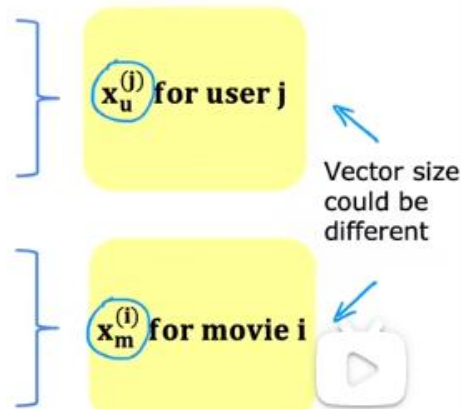
Examples of user and item features

User features:

- • Age
- • Gender (1 hot)
- • Country (1 hot, 200)
- • Movies watched (1000)
- • Average rating per genre
- • ...

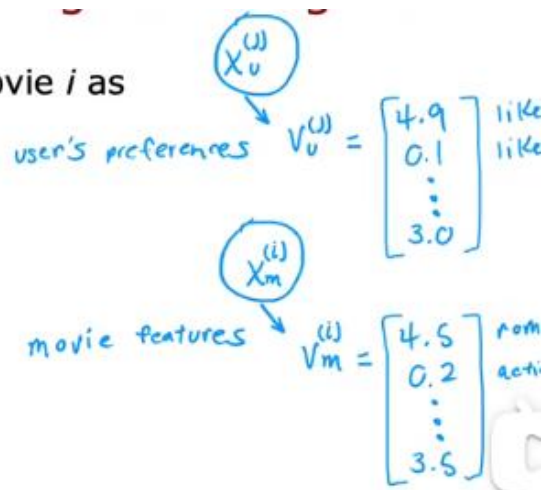
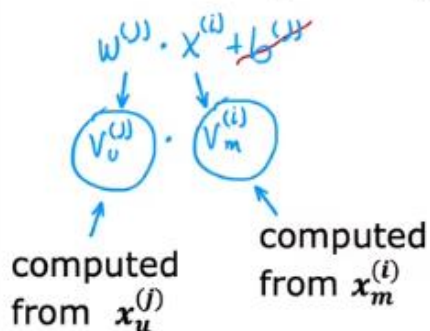
Movie features:

- • Year
- • Genre/Genres
- • Reviews
- • Average rating



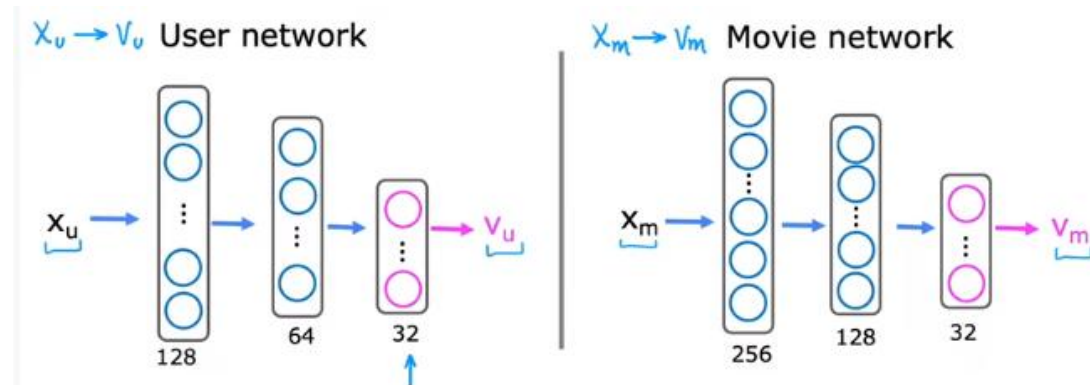
用基于内容过滤的方法推荐电影，用户和电影的特征数量可能不同，那也没关系，可以开发一种算法来学习匹配用户和电影：

Predict rating of user j on movie i as

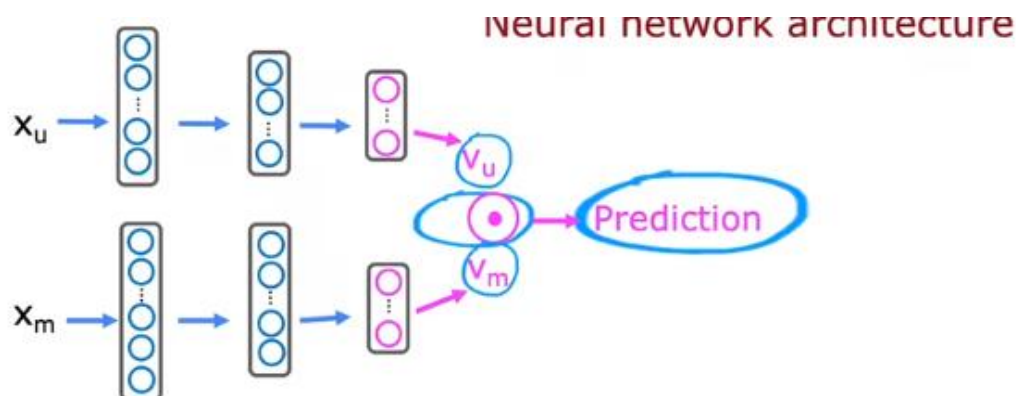


之前用 $x_u \cdot x_m + b$ 预测用户 j 对电影 i 的评分，现在把 b 去掉，事实证明这不会影响基于内容过滤的性能。然后把 x_u 和 x_m 替换，根据用户行为特征设计出系数向量 v_u ，用电影特征设计出新的向量 v_m 。这里 v_u 表示对各类电影，比如爱情、动作，的喜爱程度； v_m 表示电影的成分，各元素是这个电影有多少是动作电影，多少是爱情电影……最开始的 x 向量维度可能不同，但最后得到 v_u 和 v_m 的大小必须相同。两个向量取点积

基于内容过滤的深度学习方法：



使用神经网络将 x_u 和 x_m 转换为 v_u 和 v_m 。两个神经网络的第三个隐藏层都是 32 个单元，所以 v_u 和 v_m 都是含 32 个元素的列向量，保证了维度相同。 v_u 和 v_m 点积得到预测值，再用 sigmoid 函数计算为 1 的概率



Cost function $J = \sum_{(i,j):r(i,j)=1} (v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)})^2 + \text{NN regularization term}$

为了确定神经网络各层的参数，建立成本函数。根据得到 v_u 和 v_m 的好坏来判断网络，使用梯度下降或其他算法进行调整，使用 NN 正则化

因为确定了 v_m ，可以用平方距离找到相似的电影进行推荐

To find movies similar to movie i :

$$\|v_m^{(k)} - v_m^{(i)}\|^2 \text{ small}$$

$$\|x^{(k)} - x^{(i)}\|^2$$

Note: This can be pre-computed ahead of time

从大型目录中推荐：

一家电影院有很多电影，遍历所有电影用神经网络进行推荐的方法显然不可行，因为计算量太大。许多损失尺度推荐系统被实现为两个步骤，称为检索和排名步骤：

在检索步骤期间将生成大量似是而非的候选项目列表，这些项目列表试图覆盖很多可能向用户推荐的内容。在检索过程中，可能包含很多用户不太可能喜欢的项目，然后在排名过程中选出最好的项目推荐给用户

检索步骤：①在用户每看完 10 个电影后，选出 10 个最相似的电影。也就是对于看完电影的特征向量，找到类似的电影，让特征向量间的平方距离最小，这些会提供一些可能的初始电影；②对用户喜欢的各种类型，分别找到排名前 10 的电影加到候选项目列表。现在，可以得到一些似是而非的电影的列表进行推荐。但是，如果它包含一些用户不喜欢的选项也是可以的：检索步骤的目标是确保广泛的覆盖范围，保证其中好电影足够多就行；③最后，获取所有候选项目，删除重复项等

排名步骤：①获取检索步骤中检索到的候选项目列表，这里只有几百个电影，用学习模型进行排名，即把用户和电影的特征输入上节课的神经网络，计算预测得分，排序

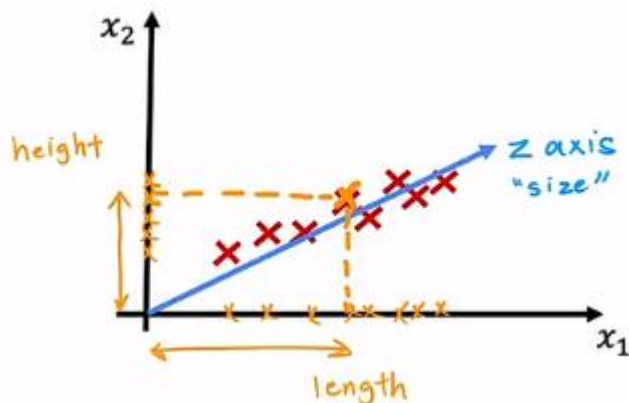
候选项目列表具体由几个电影组成要看实际的运行速度，可以是 100，也可以更多

降低特征数量：

特征数量很多时，比如 100 个，这时可视化等操作就难以实现。主成分分析（PCA）是一种算法，可以将特征数量减少，以便进行绘图和可视化

对于一辆汽车，具有长度、宽度、车轮直径等多个特征。假设特征 x_1 是汽车长度， x_2 是宽度。因为道路宽度等原因，很多地区的汽车宽度保持在一个小范围内，而长度则有很多。如果想减少特征数量，可以取 x_1 ，因为 x_2 因汽车而异（根据实际情况分析减少特征）

如果 x_1 是汽车长度， x_2 是汽车高度，往往汽车越长，汽车高度也会有一定程度升高。这里只选其中一个而忽略另外一个可能不太好，似乎两个都是有用的信息



可以在坐标系建立一个“z 轴”，这不是空间直角坐标系的 z 轴，而是与 x_1, x_2 在同一平面，轴上的每一点的坐标能同时反映 x_1 和 x_2 两个特征

PCA 的想法是找到一个或多个新轴，例如上面的 z 轴。这样，在新轴测量数据坐标时，仍然能得到有关汽车的有用的信息。PCA 通常用于减少非常大量的特征，比如 10, 20 甚至数千个特征，减少到可能只有两三个特征，以便在二维或三维视图将数据可视化

PCA 算法:

无监督学习算法。只有特征 x_1 等，没有标签 y

假如有不同的特征 x_1, x_2 , 如何找到一个轴捕获特征:

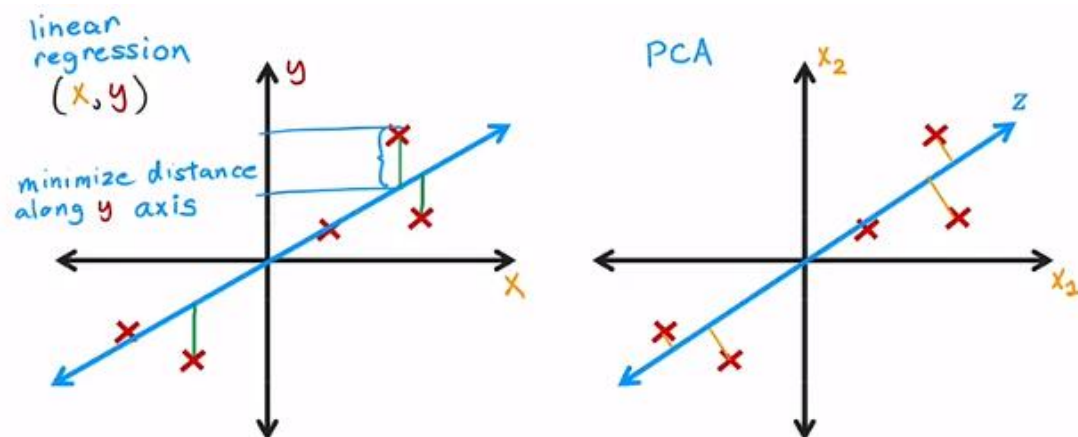
①首先要进行预处理，把特征归一化到均值为 0。如果特征尺度差异很大，必须先特征缩放。这里先减去均值，变成 0 均值，再进行缩放

②建立一个“ z 轴”，这个轴叫主成分。把数据投影到它上面时，最终会得到最大可能的方差，也就是保留信息最多的轴。（感觉轴的位置有点像线性回归，实际上不一样）

（之前说高方差可能是过拟合，这里没关系，降维时要尽可能减少信息损失）

这里把两个特征降维到一个特征。如果要添加附加轴，比如有 50 个特征并且要找 3 个主成分，得到一个轴后，第二个轴一定与第一个轴成 90 度，第三个轴垂直于前两个轴。因为正交时信息重合最少

PCA 不是线性回归。线性回归是与数据点残差有关，PCA 与数据点到轴的垂直距离有关



如果尝试预测 y 的值，使用线性回归；如果是降维，将其可视化，使用 PCA

PCA 有一步叫重建。比如 x_1, x_2 降维成 z ，只给定数字 z ，可以近似出原来的 x_1, x_2 。没有足够的信息精确返回 x_1 和 x_2 ，但是通过作垂线可以得到近似的结果

代码实现: sklearn 中使用 fit 函数来执行，它会自动进行均值归一化，减去每个特征的均值，所以你不需要单独执行归一化。运行拟合函数后得到了主成分。之后，建议查看每个新主成分在多大程度上解释了数据差异，使用解释的方差比函数确定保留了多少信息。然后对于每个训练示例，将它们投影到新轴

PCA，比如找到两个主成分，可以输出解释比率，比如 0.992 和 0.008，意味着第一个主成分解释 99.2% 的方差，第二个主成分解释 0.8% 的方差，加起来是 1，因为它们一起解释了 100% 的方差

过去，PCA 可以用于数据压缩，减少特征数量方便特征存储，不过现在数据传输变得方便就不用这个了；还可以通过减少特征来加速监督学习模型的训练，对老一代学习模型比如支持向量机有用，但是像深度学习算法很少用这个方法，更常见的是只获取高维数据集并输入神经网络

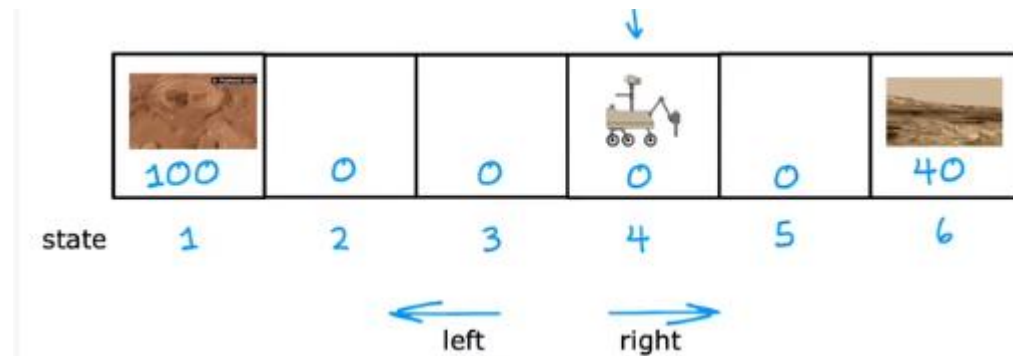
强化学习:

比如要设计一个无人驾驶直升机的程序，用监督学习的方法构建分类模型，确定在什么情况下进行什么操作，效果可能不好，因为很难得到一个实际情况与理想动作的数据集，所以我们使用强化学习

强化学习的一个关键输入是一种叫做奖励或奖励函数的东西，在直升机表现不佳时，告诉直升机什么做的好，让它得到更多好的结果和更少的坏结果。告诉它该做什么而不是如何去做，指定奖励函数而不是最佳动作，可以让设计系统时更灵活

做得好的时候给一点奖励，比如+1，做的不好时给一个很大的负奖励，比如-1000，这会激励直升机在大多数时间表现良好。过程中不会告诉它每个输入的正确输出，而是指定奖励函数告诉它什么时候做的好或不好，让它自己选择好的动作

以火星探测器为例：

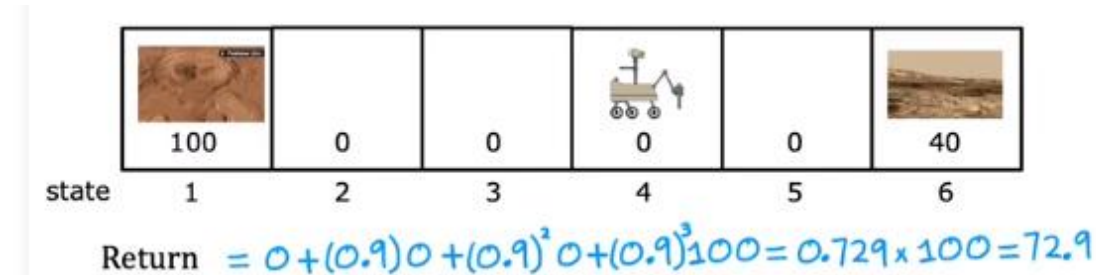


探测器的位置称为状态，初始位置是状态 4。向左到达 3 和 2 得到 0 奖励，到达终端状态 1 或 6 时一天结束，也就是在该状态得到奖励后不能再移动，所以它最终只能得到 100 或 40 的奖励。

状态、奖励和下一个状态，构成强化学习的核心要素，表示为 $(s, a, R(s), s')$ ，分别是状态、选择的动作、奖励、新的状态。

强化学习的回报：

回报是各种奖励因素的总和，但由一个附加因素加权，称为折扣因子，折扣因子是一个略小于 1 的系数，比如选择 0.9。第一步往后，系数为折扣因子的幂运算



更一般的回报公式是，如果你的机器人经历了一些状态并在第一步获得奖励 R_1 ，第二步 R_2 ，以此类推。折扣因子是 γ ，那回报就是：

$$\text{Return} = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$$

像这样一直加到 $\text{pow}(\gamma, n-1) * R_n$

折扣因子 γ 的作用是，强化学习模型有点不耐烦，因为获得的奖励逐渐变少，导致总回报的价值越高

对于具有负奖励的系统，它会导致算法尝试推送，尽可能将奖励放在未来

决策：强化学习中的策略：

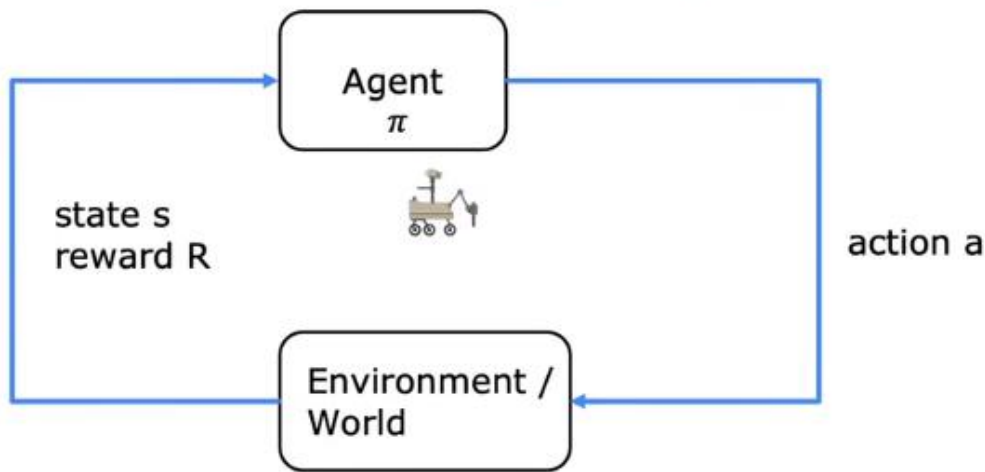
可以决定追求最接近的奖励，离哪个奖励近就去哪个；也可以追求最大的奖励；或者总是争取较小的奖励，这可能不是一个好主意，但也是另一种选择；也可以选择去最大的奖励，除非距离较小的奖励仅一步之遥

我们的目标是提出一个称为策略 π 的函数，将任何工作输入并对其进行映射，得到任何状态下希望采取的行动

审查关键概念：

	Mars rover 	Helicopter 	Chess 
states	6 states	position of helicopter	pieces on board
actions		how to move control stick	possible move
rewards	100, 0, 40	+1, -1000	+1, 0, -1
discount factor γ	0.5	0.99	0.995
return	$R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$	$R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$	$R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$
policy π		Find $\pi(s) = a$ 	Find $\pi(s) = a$ 

这种程序也叫马尔可夫决策过程（MDP），指的是未来只取决于当前状态而不取决于你是怎么到这里的，不取决于任何事物



状态-动作价值函数：

通常用 Q 表示。是你可能处于的状态以及你可能选择在该状态和 $Q(s, a)$ 中采取的行动的函数。 $Q(s, a)$ 将给出一个等于回报的数字。如果从状态 s 开始，并只采取行动 a 一次，在采取行动 a 后，你就会表现得最佳。因此，在那之后，你采取的任何行动都会导致最高的回报

100	50	25	12.5	20	40
100	0	0	0	0	40

← return
← action
← reward

这个例子中，当火星探测器位于状态 2, 3, 4 就会向左走，位于状态 5 就会向右走。取 γ 为 0.5， $Q(s, a)$ 将等于总回报。如果你说采取行动 a 开始，然后在此之后表现最佳，意思是根据本政策采取行动

	50	12.5			
100	0	0	0	0	40
1	2	3	4	5	6

实际上就是先选一个状态，通过计算判断最佳路径。比如从状态 2 开始向左是最佳，如果先向右再向左，显然不如一开始就向左。 Q 函数计算各种情况的回报，找到最佳路径（有点像 dijkstra？）

贝尔曼方程：

用于计算状态动作值函数

$Q(s, a)$ = Return if you

- start in state s .
- take action a (once).
- then behave optimally after that.

			←		
1	2	3	4	5	6

$R(1)=100$ $R(2)=0$... $R(6)=40$

s : current state
 a : current action

$R(s)$ = reward of current state

s' : state you get to after taking action a
 a' : action that you take in state s'

贝尔曼方程表示为：

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a')$$

$R(s)$ 是回报， γ 是折扣因子，乘上状态为 s' 情况下选择行动方式 a' 时的最大状态动作价值，得到 $Q(s, a)$ 。比如：

100	100	50	12.5	25	6.25	12.5	10	6.25	20	40	40
100	0	0	0	0						40	
1	2	3	4	5	6						

$s = 2$
 $a = \rightarrow$
 $s' = 3$

$$Q(2, \rightarrow) = R(2) + 0.5 \max_{a'} Q(3, a')$$

$$= 0 + (0.5)25 = 12.5$$

$$Q(4, \leftarrow) = R(4) + 0.5 \max_{a'} Q(3, a')$$

$$= 0 + (0.5)25 = 12.5$$

$s = 4$
 $a = \leftarrow$
 $s' = 3$

如果位于终端状态, $Q(s, a) = R(s)$, 因为 s' 不存在, 第二项消失
原理就是:

$$\text{Return} = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$$

random stochastic environment:

实践中, 机器人不一定能完全按我们的意愿行动, 比如碰上斜坡然后滑到错误的地方。强化学习中, 可以模拟随机环境

命令一个机器人往左走, 它大概率会向左走, 比如 0.9 的概率, 也有可能因为滑倒什么的往右走, 概率是 0.1

随机强化学习中, 我们感兴趣的不是最大化回报, 因为那是一个随机数。我们感兴趣的是最大化折扣奖励总和的平均值。比如按设置的方法试 1000 次, 得到许多不同的奖励序列, 对它们取平均, 这就是我们说的预期回报

$$\begin{aligned} \text{Expected Return} &= \text{Average}(R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots) \\ &= E[R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots] \end{aligned}$$

可以稍微修改贝尔曼方程, 区别在于, 新的方程在 s 状态采取行动 a 时, 下一个状态随机

$$Q(s, a) = R(s) + \gamma E[\max_{a'} Q(s', a')]$$

连续状态空间:

上面例子中, 火星探测器的位置是离散值, 实际上, 探测器可以位于任何一个位置, 是连续状态空间, 1 到 6 之间的任何实数都有可能。状态可能不仅包括一个数字, 这里 s 可以用向量 $[x \ y \ \theta \ x' \ y' \ \theta']$ 表示, 为 x 位置, y 位置, 角度, 以及速度 (前面三个变量求导)

(用自由度与相应变量的导数来描述?)

以登月器为例: 要让登月器时刻调节方向等, 落到指定位置, 防止坠毁。在上面 s 的基础上加入 l 和 r , 表示登月器左脚和右脚是否着地, 用 0 和 1 表示

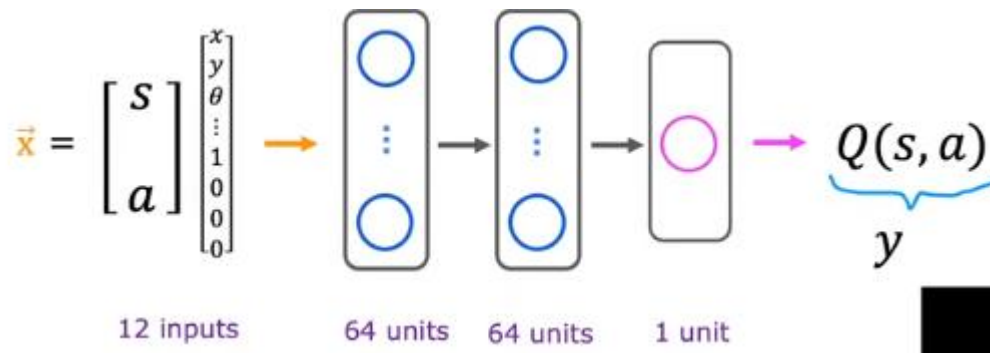
- Getting to landing pad: 100 - 140
- Additional reward for moving toward/away from pad.
- Crash: -100
- Soft landing: +100
- Leg grounded: +10
- Fire main engine: -0.3
- Fire side thruster: -0.03

对此设置一个中等复杂的奖励函数。后面两个对主引擎和左右推进器的启动进行惩罚, 鼓励登月器节省燃料。根据这个奖励函数, 想办法让回报最大

学习状态值函数:

学习算法的核心是建立一个神经网络，输入当前状态和当前动作，计算或逼近 $Q(s, a)$ 。对于登月器，采用状态 s 和任意动作的 a ，并把它们放在一起。状态是之前那个数字列表 $[x \ y \ \theta \ x' \ y' \ \theta' \ l \ r]$ 。最后，我们有四种可能的动作：nothing, left, main, a main engine, right。对四种动作中的任何一个进行编码，使用 one-hot 特征向量：比如是第一个动作，那就用 1,0,0,0 进行编码；第二个动作那就是 0,1,0,0

神经网络的输入是状态和动作，包括前面的数字列表和动作编码，共 12 个数，称为 x 。训练神经网络进行近似，得到输出 y 为近似的 $Q(s, a)$



可以使用贝尔曼方程创造大量数据集进行监督学习

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a')$$

x y

$$f_{w,b}(x) \approx y$$

$$y^{(1)} = R(s^{(1)}) + \gamma \max_{a'} Q(s'^{(1)}, a')$$

$(s, a, R(s), s')$

$(s^{(1)}, a^{(1)}, R(s^{(1)}), s'^{(1)}) \leftarrow$

$(s^{(2)}, a^{(2)}, R(s^{(2)}), s'^{(2)}) \leftarrow$

$(s^{(3)}, a^{(3)}, R(s^{(3)}), s'^{(3)}) \leftarrow$

x	y
$x^{(1)} = (s^{(1)}, a^{(1)})$	$y^{(1)}$

状态一直在变。上图中下边的每一个元组都是一个状态和动作，都能够训练一个神经网络。每一步都训练一次，找到每一步的最优解。（有点像动态规划？）

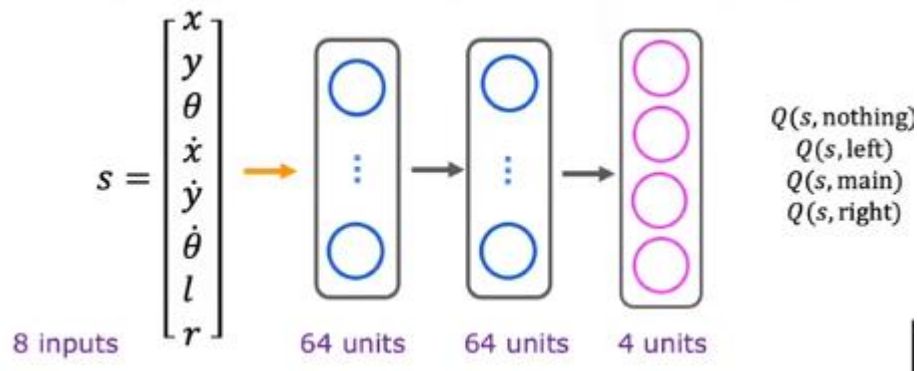
参数可以先随机初始化，根据结果不断改良

算法改进：

可以改进神经网络架构：

In a state s , use neural network to compute
 $Q(s, \text{nothing}), Q(s, \text{left}), Q(s, \text{main}), Q(s, \text{right})$
 Pick the action a that maximizes $Q(s, a)$

原来的神经网络中，输入特征后，要分别计算动作为 nothing, left, main 和 right 四种情况时的 Q ，然后才能找到最大的 Q 。可以改进神经网络，同时输出这四个值



改成使用四个输出单元同时输出四个值，计算效率更高

也可以使用 epsilon-贪婪策略：

根据当前的 s 选择能够最大化 Q 的 a 。（贪心算法？）

这种方法计算效率更高，但是从全局上看可能不是最好的方法

为了跳出局部最优，给予 0.95 的概率选择能够最大化 Q 的动作，有 0.05 的概率随机选择别的动作（epsilon-greedy 策略）（有点像模拟退火？）

也可以进行小批量和软更新：

梯度下降时，如果训练集很大，比如有 10000 个数据，可以只选择 1000 个主要示例的数据子集。每次迭代只看一个数据子集（每次的都不一样，比如先看前 10%，然后看 10%到 20%，以此类推），而不是所有数据，迭代运行更快。

小批量梯度下降有时可能不会按趋近全局最小的方向运作，但是计算成本要低很多。

训练集非常大时，监督学习算法常把小批量梯度下降与 adam 算法等结合，这比批量梯度下降更常见。强化学习中也是：训练集很大时，每次只取一部分数据训练神经网络

对 Q 的参数进行更新时，直接把 w, b 换成新的，可能导致性能突然变差。为了解决这个问题，更新时把新旧参数加权相加，称为软更新

$$\text{Set } Q = Q_{\text{new}}.$$

$\uparrow$ w, b $\uparrow$ $w_{\text{new}}, b_{\text{new}}$

$$W = 0.01 W_{\text{new}} + 0.99 W$$

$$B = 0.01 B_{\text{new}} + 0.99 B$$

以原来的参数为主，利用新参数进行更新。权重根据实际调整

软更新方法让强化学习算法更可靠地收敛，使算法不太可能振荡或转向或具有其他不良特性